

|                 |                                   |                                                                                                                     |   |              |            |
|-----------------|-----------------------------------|---------------------------------------------------------------------------------------------------------------------|---|--------------|------------|
|                 |                                   | <b>Escuela Politécnica Superior</b><br><b>Ingeniería Informática</b><br><b>Prácticas de Sistemas Informáticos 2</b> |   |              |            |
| <b>Grupo</b>    | <b>2313</b>                       | <b>Práctica</b>                                                                                                     | 2 | <b>Fecha</b> | 22/02/2026 |
| <b>Alumno/a</b> | Rangel, Urdaneta, Sebastián Jesús |                                                                                                                     |   |              |            |
| <b>Alumno/a</b> | Dal, Nieto, Angel Fernando        |                                                                                                                     |   |              |            |

## Práctica 1

### *Ejercicio número 1:*

Ejecute los pasos descritos anteriormente, uno tras otro, para crear el nuevo proyecto. Incluya en la memoria evidencias (capturas de pantalla) de haber realizado estos pasos. Indica por qué no será necesario hacer uso de los formularios Django y las plantillas en la aplicación servidor RPC.

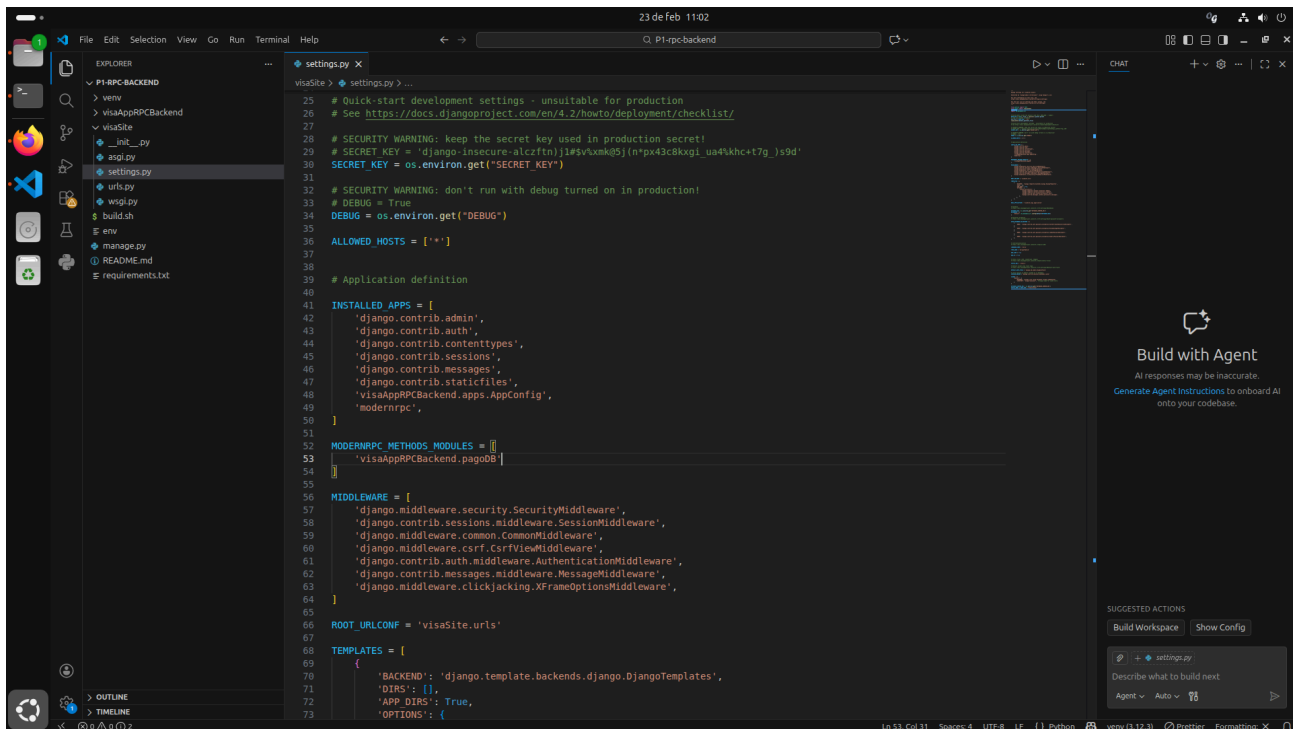
En la aplicación servidor RPC, no es necesario el uso de formularios Django porque no es su función interactuar directamente con el usuario a través del navegador web. En lugar de esto, el servidor actúa como proveedor de servicios que expone lógica de negocio y acceso a datos mediante procedimientos remotos.

Pasos descritos del enunciado:

```
e504937@6A-2-6-2: ~/Descargas/SI2
(venv) e504937@6A-2-6-2:~/Descargas/SI2/P1-rpc-backend$ ls
build.sh  env  manage.py  README.md  requirements.txt  venv  visaAppRPCBackend  visaSite
(venv) e504937@6A-2-6-2:~/Descargas/SI2/P1-rpc-backend$ code .
(venv) e504937@6A-2-6-2:~/Descargas/SI2/P1-rpc-backend$ ls
build.sh  env  manage.py  README.md  requirements.txt  venv  visaAppRPCBackend  visaSite
(venv) e504937@6A-2-6-2:~/Descargas/SI2/P1-rpc-backend$ cd ..
(venv) e504937@6A-2-6-2:~/Descargas/SI2$ ls
P1-base  P1-rpc-backend
(venv) e504937@6A-2-6-2:~/Descargas/SI2$
```

```
e504937@6A-2-6-2: ~/Descargas/SI2/P1-rpc-backend/visaAppRPCBackend
(venv) e504937@6A-2-6-2:~/Descargas/SI2/P1-rpc-backend$ ls
build.sh  env  manage.py  README.md  requirements.txt  venv  visaAppRPCBackend  visaSite
(venv) e504937@6A-2-6-2:~/Descargas/SI2/P1-rpc-backend$ cd visaAppRPCBackend/
(venv) e504937@6A-2-6-2:~/Descargas/SI2/P1-rpc-backend/visaAppRPCBackend$ ls
admin.py  apps.py  __init__.py  management  migrations  models.py  pagoDB.py  urls.py
(venv) e504937@6A-2-6-2:~/Descargas/SI2/P1-rpc-backend/visaAppRPCBackend$
```

Fichero settings.py:



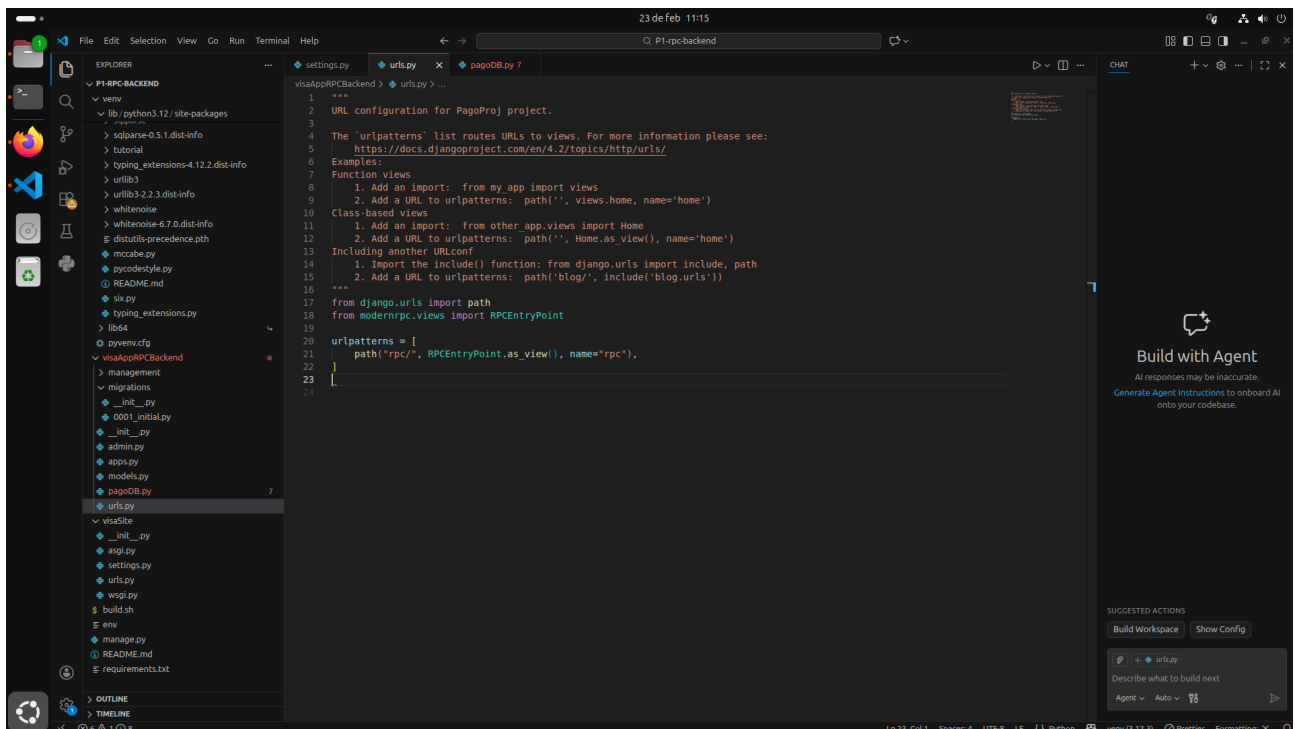
```
25 # Quick-start development settings - unsuitable for production
26 # See https://docs.djangoproject.com/en/4.2/howto/deployment/checklist/
27
28 # SECURITY WARNING: keep the secret key used in production secret!
29 # SECRET_KEY = 'django-insecure-rlczftn)1isv/xmkq5j(n*px43c8kxgl_ua4k4hc+7g_)s9d!'
30 SECRET_KEY = os.environ.get('SECRET_KEY')
31
32 # SECURITY WARNING: don't run with debug turned on in production!
33 # DEBUG = True
34 DEBUG = os.environ.get('DEBUG')
35
36 ALLOWED_HOSTS = ['*']
37
38
39 # Application definition
40
41 INSTALLED_APPS = [
42     'django.contrib.admin',
43     'django.contrib.auth',
44     'django.contrib.contenttypes',
45     'django.contrib.sessions',
46     'django.contrib.messages',
47     'django.contrib.staticfiles',
48     'visaAppRPCBackend.apps.AppConfig',
49     'modernrpc',
50 ]
51
52 MODERNRPC_METHODS_MODULES = [
53     'visaAppRPCBackend.pagoOB'
54 ]
55
56 MIDDLEWARE = [
57     'django.middleware.security.SecurityMiddleware',
58     'django.contrib.sessions.middleware.SessionMiddleware',
59     'django.middleware.common.CommonMiddleware',
60     'django.middleware.csrf.CsrfViewMiddleware',
61     'django.contrib.auth.middleware.AuthenticationMiddleware',
62     'django.contrib.messages.middleware.MessageMiddleware',
63     'django.middleware.clickjacking.XFrameOptionsMiddleware',
64 ]
65
66 ROOT_URLCONF = 'visaSite.urls'
67
68
69 TEMPLATES = [
70     {
71         'BACKEND': 'django.template.backends.django.DjangoTemplates',
72         'DIRS': [],
73         'APP_DIRS': True,
```

## Ejercicio número 2:

Ejecute los pasos descritos anteriormente, uno tras otro, para exportar la funcionalidad de acceso a la BD como procedimientos remotos. Incluya en la memoria evidencias (capturas de pantalla) de haber realizado estos pasos. Indica razonadamente por qué es necesario hacer uso del método `model_to_dict` tras registrar un pago.

Se utiliza el método `model_to_dict` tras registrar un pago para transformar un objeto complejo de base de datos en una estructura de datos simple y compatible que el servidor RPC puede transmitir al cliente.

## Fichero visaAppRPCBackend/urls.py:

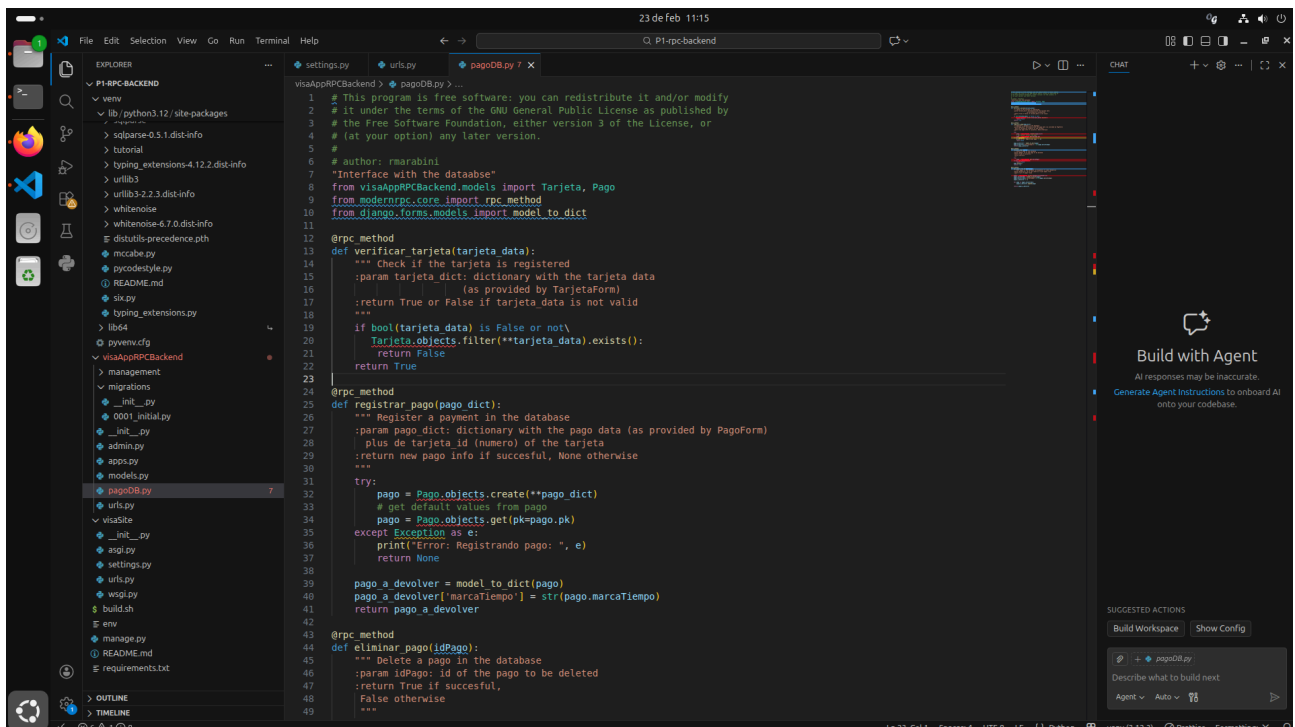


The screenshot shows the Visual Studio Code editor with the file `urls.py` open in the `visaAppRPCBackend` project. The file contains the following code:

```
1 # -*- coding: utf-8 -*-
2 """
3 URL configuration for PagaProj project.
4
5 The 'urlpatterns' list routes URLs to views. For more information please see:
6     https://docs.djangoproject.com/en/4.2/topics/http/urls/
7 Examples:
8 Function views
9     1. Add an import: from my_app import views
10    2. Add a URL to urlpatterns: path('', views.home, name='home')
11 Class-based views
12     1. Add an import: from other_app.views import Home
13     2. Add a URL to urlpatterns: path('', Home.as_view(), name='home')
14 Including another URLconf
15     1. Import the include() function: from django.urls import include, path
16     2. Add a URL to urlpatterns: path('blog/', include('blog.urls'))
17 """
18 from django.urls import path
19 from modernrpc.views import RPCEntryPoint
20
21 urlpatterns = [
22     path('rpc/', RPCEntryPoint.as_view(), name='rpc'),
23 ]
```

The Explorer sidebar on the left shows the project structure, with `urls.py` selected under the `visaAppRPCBackend` folder. The right sidebar shows the "Build with Agent" panel.

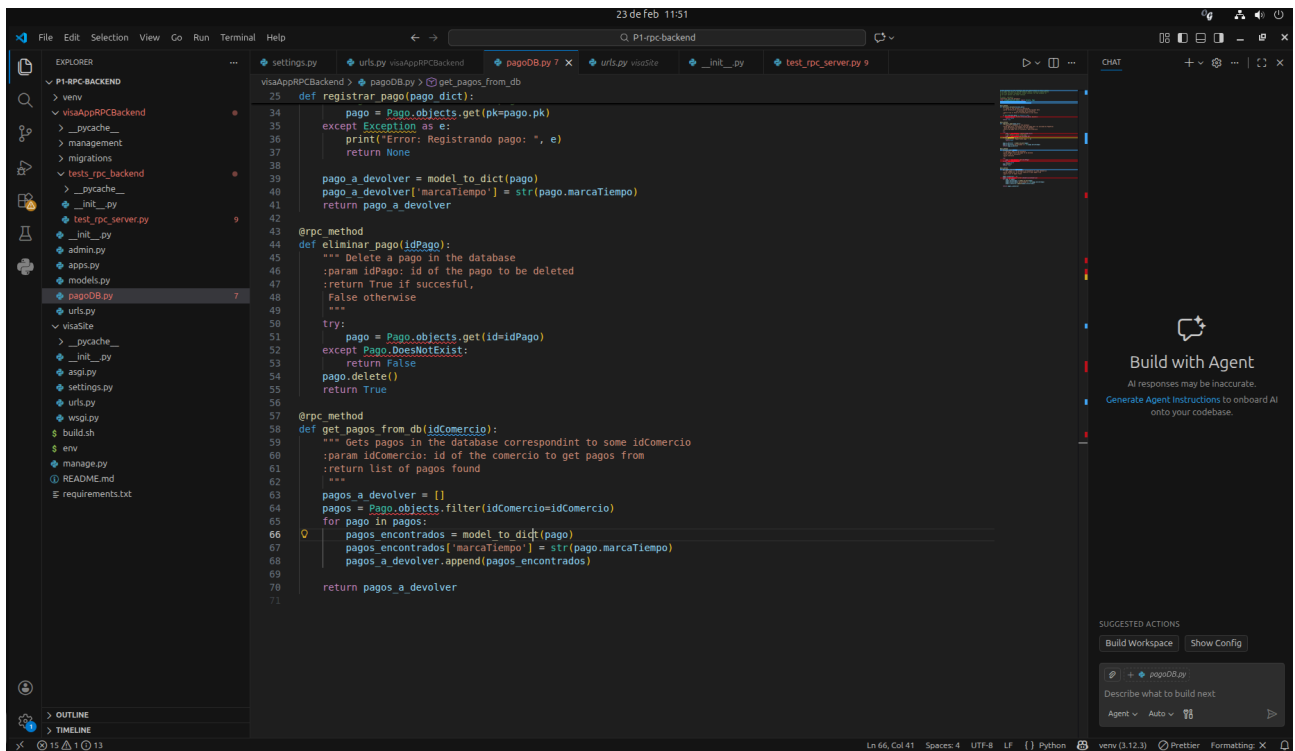
## Fichero padoDB.py:



The screenshot shows the Visual Studio Code editor with the file `padoDB.py` open in the `visaAppRPCBackend` project. The file contains the following code:

```
1 # This program is free software: you can redistribute it and/or modify
2 # it under the terms of the GNU General Public License as published by
3 # the Free Software Foundation, either version 3 of the License, or
4 # (at your option) any later version.
5
6 # author: rmarabini
7
8 """Interface with the database"""
9 from visaAppRPCBackend.models import Tarjeta, Pago
10 from modernrpc.core import rpc_method
11 from django.forms.models import model_to_dict
12
13 @rpc_method
14 def verificar_tarjeta(tarjeta_data):
15     """ Check if the tarjeta is registered
16     :param tarjeta_dict: dictionary with the tarjeta data
17                         (as provided by TarjetaForm)
18     :return True or False if tarjeta_data is not valid
19     """
20     if bool(tarjeta_data) is False or not\
21         Tarjeta.objects.filter(**tarjeta_data).exists():
22         return False
23     return True
24
25 @rpc_method
26 def registrar_pago(pago_dict):
27     """ Register a payment in the database
28     :param pago_dict: dictionary with the pago data (as provided by PagoForm)
29     :param tarjeta_id (numero) of the tarjeta
30     :return new pago info if successful, None otherwise
31     """
32     try:
33         pago = Pago.objects.create(**pago_dict)
34         # get default values from pago
35         pago = Pago.objects.get(pk=pago.pk)
36     except Exception as e:
37         print("Error: Registrando pago: ", e)
38         return None
39
40     pago_a_devolver = model_to_dict(pago)
41     pago_a_devolver['marcaTiempo'] = str(pago.marcaTiempo)
42     return pago_a_devolver
43
44 @rpc_method
45 def eliminar_pago(idPago):
46     """ Delete a pago in the database
47     :param idPago: id of the pago to be deleted
48     :return True if successful,
49             False otherwise
50     """
```

The Explorer sidebar on the left shows the project structure, with `padoDB.py` selected under the `visaAppRPCBackend` folder. The right sidebar shows the "Build with Agent" panel.



### Ejercicio número 3:

Ejecuta los pasos descritos más arriba e incluye evidencias en la memoria de haberlos llevado a cabo. Ejecuta los test proporcionados con el proyecto y comprueba que no devuelven errores. Adjunta en la memoria una captura de pantalla en la que se muestre el resultado de ejecutar los test (python manage.py test visaAppRPCBackend.tests\_rpc\_backend). Los test proporcionados deben tomarse como requisitos extras del sistema.

VM1 corriendo con la BBDD si2db:

```
si2@si2-ubuntu-vm-1:~$ sudo -u postgres psql
psql (16.10 (Ubuntu 16.10-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# DROP DATABASE si2db;
DROP DATABASE
postgres=# CREATE DATABASE si2db OWNER alunnodb;
CREATE DATABASE
postgres=# \c si2db alunnodb
You are now connected to database "si2db" as user "alunnodb".
si2db=# \dt
Did not find any relations.
si2db=# \dt

          List of relations
Schema | Name                | Type  | Owner
-----|-----|-----|-----
public | auth_group           | table | alunnodb
public | auth_group_permissions | table | alunnodb
public | auth_permission      | table | alunnodb
public | auth_user            | table | alunnodb
public | auth_user_groups     | table | alunnodb
public | auth_user_user_permissions | table | alunnodb
public | django_admin_log     | table | alunnodb
public | django_content_type  | table | alunnodb
public | django_migrations   | table | alunnodb
public | django_session       | table | alunnodb
public | pago                 | table | alunnodb
public | tarjeta              | table | alunnodb
(12 rows)

si2db=# SELECT * FROM tarjeta LIMIT 10;
si2db=#
```

En el host:

```
(venv) e50493706A-2-6-2:~/Descargas/SI2/P1-rpc-backend$ python3 manage.py makemigrations
No changes detected
(venv) e50493706A-2-6-2:~/Descargas/SI2/P1-rpc-backend$ python3 manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions, visaAppRPCBackend
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
  Applying visaAppRPCBackend.0001_initial... OK
(venv) e50493706A-2-6-2:~/Descargas/SI2/P1-rpc-backend$ python3 manage.py populate
Database cleaned successfully
Tarjeta objects created successfully
```

Devuelve error HTTP 405 tal como dice el enunciado:

```
^C(venv) e50493706A-2-6-2:~/Descargas/SI2/P1-rpc-backend$ python3 manage.py runserver 8001
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
February 23, 2026 - 11:35:23
Django version 5.2.2, using settings 'visaSite.settings'
Starting development server at http://127.0.0.1:8001/
Quit the server with CONTROL-C.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/5.2/howto/deployment/
[23/Feb/2026 11:35:30] "GET /visaAppRPCBackend/rpc HTTP/1.1" 301 0
Method Not Allowed (GET): /visaAppRPCBackend/rpc/
Method Not Allowed: /visaAppRPCBackend/rpc/
[23/Feb/2026 11:35:30] "GET /visaAppRPCBackend/rpc/ HTTP/1.1" 405 0
█
```

Resultado de los tests:

```
(venv) e50493706A-2-6-2:~/Descargas/SI2/P1-rpc-backend$ python manage.py test visaAppRPCBackend.tests_rpc_backend
Found 4 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 4 tests in 0.048s

OK
Destroying test database for alias 'default'...
(venv) e50493706A-2-6-2:~/Descargas/SI2/P1-rpc-backend$ █
```

## Ejercicio número 4:

Ejecuta los pasos descritos más arriba. Prueba a acceder desde el navegador del PC del laboratorio a la URL `localhost:28000/visaAppRPCBackend/rpc` para comprobar que el despliegue de la aplicación es correcto. Incluye evidencias en la memoria en forma de capturas de pantalla mostrando que el acceso a dicha URL es correcto.

En el Host:

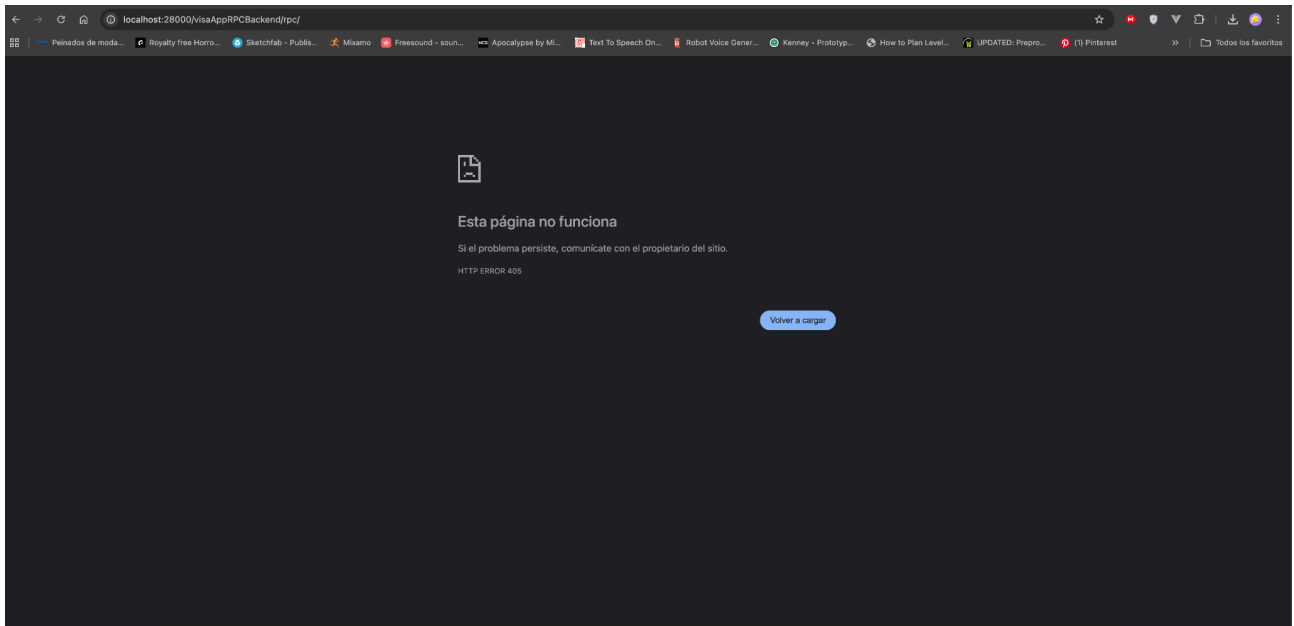
```
(venv) sebastianrangel@iMac-de-Sebastian P1-rpc-backend % curl -X POST http://localhost:28000/visaAppRPCBackend/rpc/
Unable to handle your request, the Content-Type header is mandatory to allow server to determine which handler can interpret your request..
(venv) sebastianrangel@iMac-de-Sebastian P1-rpc-backend % curl -X POST http://localhost:28000/visaAppRPCBackend/rpc/ \
-H "Content-Type: text/xml" \
-d "<methodCall><methodName>system.listMethods</methodName><params></params></methodCall>"
<?xml version="1.0"?>
  <methodResponse>
    <params>
      <param>
        <value><array><data>
          <value><string>eliminar_pago</string></value>
          <value><string>get_pagos_from_db</string></value>
          <value><string>registrar_pago</string></value>
          <value><string>system.listMethods</string></value>
          <value><string>system.methodHelp</string></value>
          <value><string>system.methodSignature</string></value>
          <value><string>system.multicall</string></value>
          <value><string>verificar_tarjeta</string></value>
        </data></array></value>
      </param>
    </params>
  </methodResponse>
```

En la VM2:

```
si2@si2-ubuntu-vm-2:~/repo/plbase/P1-rpc-backend/visaSite$ sudo systemctl status gunicorn
● gunicorn.service - Gunicorn WSGI Application Server
   Loaded: loaded (/etc/systemd/system/gunicorn.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-02-23 17:53:48 CET; 5s ago
     Main PID: 2877 (gunicorn)
       Tasks: 2 (limit: 1670)
      Memory: 47.2M (peak: 47.5M)
         CPU: 320ms
    CGroup: /system.slice/gunicorn.service
            └─2877 /home/si2/venv/bin/python3 /home/si2/venv/bin/gunicorn --workers 1 --bind 0.0.0.0:8000 visaSite.wsgi:application
              └─2879 /home/si2/venv/bin/python3 /home/si2/venv/bin/gunicorn --workers 1 --bind 0.0.0.0:8000 visaSite.wsgi:application

feb 23 17:53:48 si2-ubuntu-vm-2 systemd[1]: Started gunicorn.service - Gunicorn WSGI Application Server.
feb 23 17:53:48 si2-ubuntu-vm-2 gunicorn[2877]: /home/si2/venv/lib/python3.12/site-packages/gunicorn/util.py:25: UserWarning: pkg_resources
feb 23 17:53:48 si2-ubuntu-vm-2 gunicorn[2877]:   import pkg_resources
feb 23 17:53:48 si2-ubuntu-vm-2 gunicorn[2877]: [2026-02-23 17:53:48 +0100] [2877] [INFO] Starting gunicorn 20.1.0
feb 23 17:53:48 si2-ubuntu-vm-2 gunicorn[2877]: [2026-02-23 17:53:48 +0100] [2877] [INFO] Listening at: http://0.0.0.0:8000 (2877)
feb 23 17:53:48 si2-ubuntu-vm-2 gunicorn[2877]: [2026-02-23 17:53:48 +0100] [2877] [INFO] Using worker: sync
feb 23 17:53:48 si2-ubuntu-vm-2 gunicorn[2879]: [2026-02-23 17:53:48 +0100] [2879] [INFO] Booting worker with pid: 2879
lines 1-18/18 (END)
```

## Respuesta del browser:



Se ha desplegado la aplicación en la VM2 utilizando Gunicorn y se ha verificado su funcionamiento mediante el puerto mapeado 28000. Al acceder vía navegador (GET), el servidor responde correctamente con un error de método no permitido, confirmando que el servicio está activo.

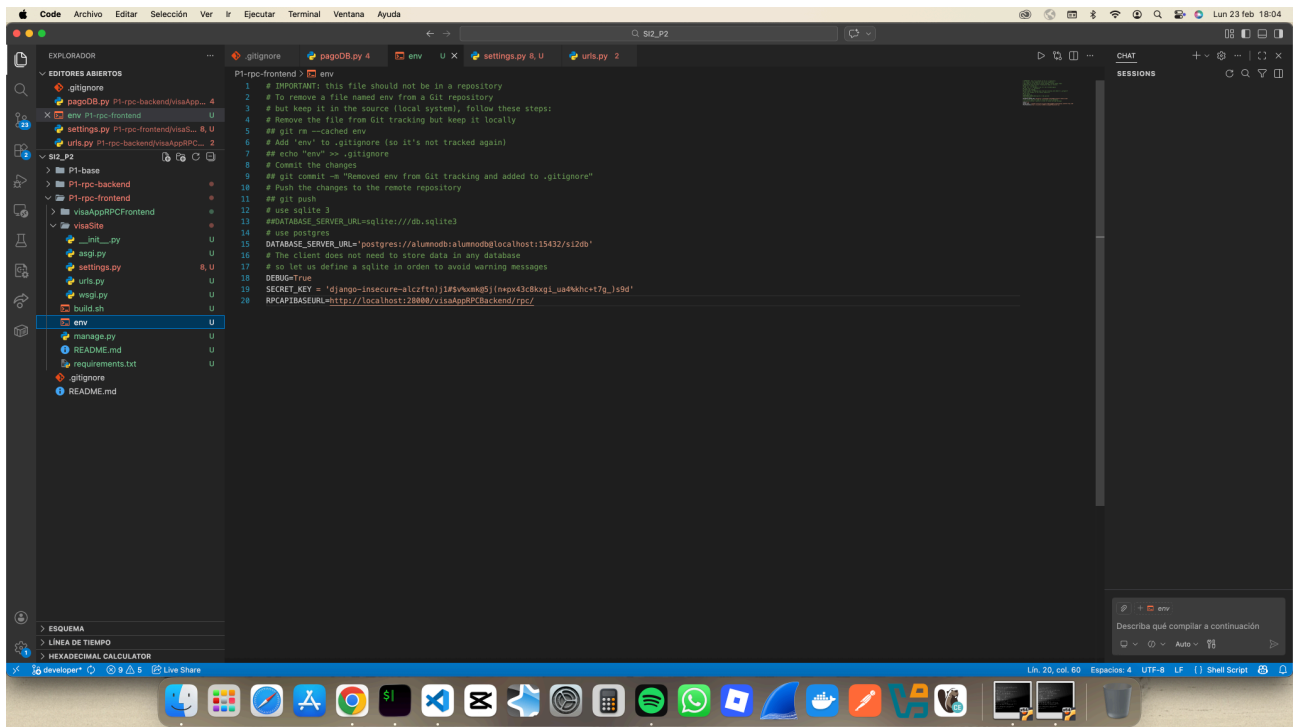
## ***Ejercicio número 5:***

**Ejecute los pasos descritos anteriormente, uno tras otro, para crear el nuevo proyecto. Incluya en la memoria evidencias (capturas de pantalla) de haber realizado estos pasos. Indica por qué no será necesario hacer uso de los modelos de datos Django en la aplicación cliente RPC.**

Pasos descritos en el enunciado:

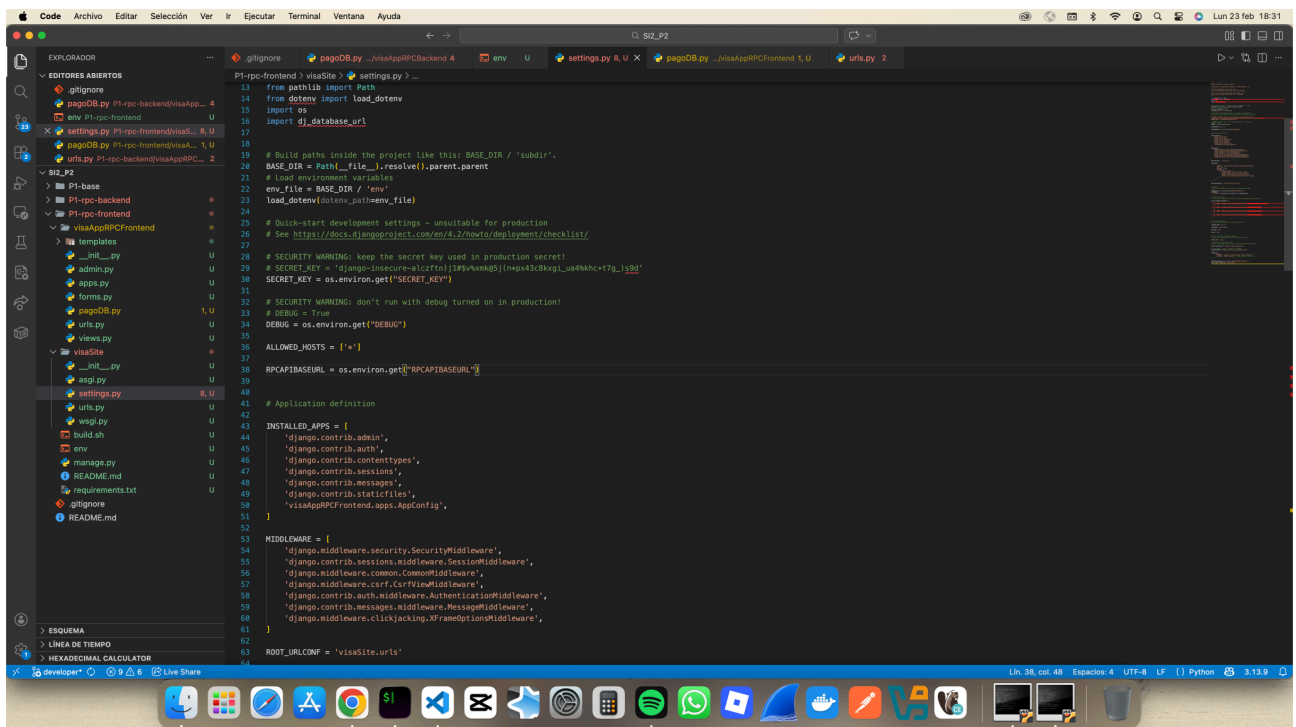
```
(env) sebastianrangel@iMac-de-Sebastian SI2_P2 % cp -r P1-base P1-rpc-frontend
(env) sebastianrangel@iMac-de-Sebastian SI2_P2 % cd P1-rpc-frontend
(env) sebastianrangel@iMac-de-Sebastian P1-rpc-frontend % mv visaApp visaAppRPCFrontend
(env) sebastianrangel@iMac-de-Sebastian P1-rpc-frontend % find ./ -type f -exec sed -i "" "s/visaApp/visaAppRPCFrontend/g" {} \;
(env) sebastianrangel@iMac-de-Sebastian P1-rpc-frontend % rm -rf visaAppRPCFrontend/models.py
(env) sebastianrangel@iMac-de-Sebastian P1-rpc-frontend % rm -rf visaAppRPCFrontend/migrations
(env) sebastianrangel@iMac-de-Sebastian P1-rpc-frontend % rm -rf visaAppRPCFrontend/management
(env) sebastianrangel@iMac-de-Sebastian P1-rpc-frontend % rm -rf visaAppRPCFrontend/tests_models.py
(env) sebastianrangel@iMac-de-Sebastian P1-rpc-frontend % rm -rf visaAppRPCFrontend/tests_views.py
```

## Fichero env:



```
1 # IMPORTANT: this file should not be in a repository
2 # To remove a file named env from a Git repository
3 # but keep it in the source (local system), follow these steps:
4 # Remove the file from Git tracking but keep it locally
5 # git rm --cached env
6 # Add 'env' to .gitignore (so it's not tracked again)
7 # echo "env" >> .gitignore
8 # Commit the changes
9 # git commit -m "Removed env from Git tracking and added to .gitignore"
10 # Push the changes to the remote repository
11 # git push
12 # use sqlite3
13 # DATABASE_SERVER_URL=sqlite:///db.sqlite3
14 # use postgres
15 DATABASE_SERVER_URL='postgres://a:ummod@a:ummod@localhost:5432/s12db'
16 # The client does not need to store data in any database
17 # so let us define a sqlite in order to avoid warning messages
18 DEBUG=True
19 SECRET_KEY = 'django-insecure-alcftnjl1f9vuwkq5j1np43c8kxgl_u4k4hc+7g_l9d'
20 RPCAPIBASEURL='http://localhost:28888/visaAppRPCBackend/rpc/'
```

## Fichero P1-rpc-frontend/visaSite/settings.py:



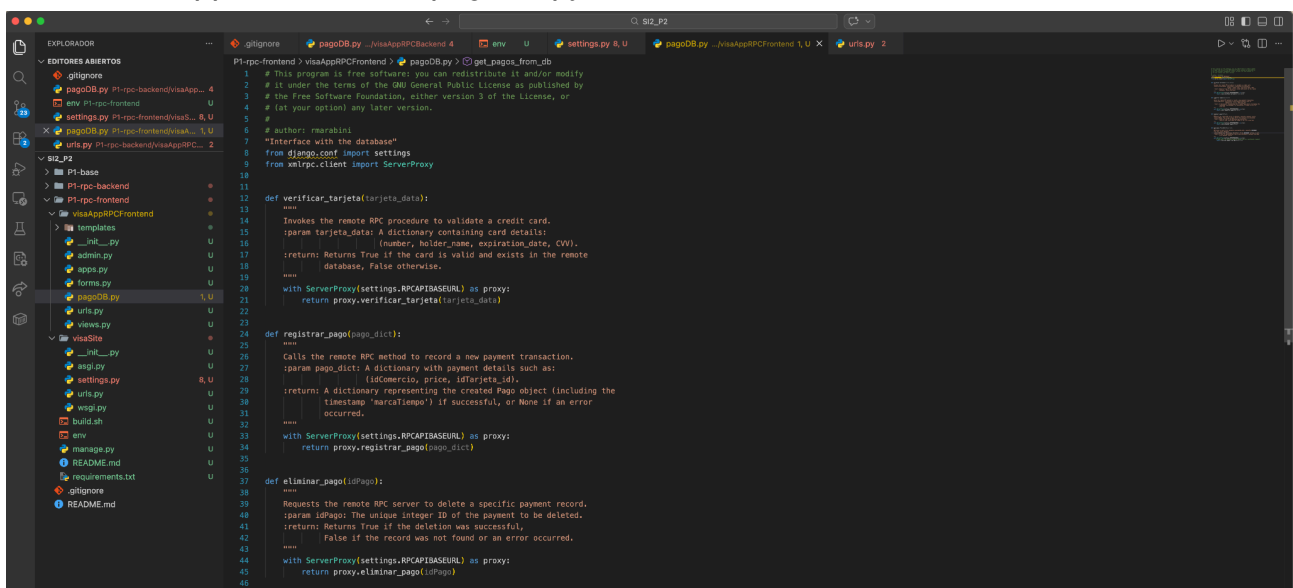
```
13 from pathlib import Path
14 from django import load_dotenv
15 import os
16 import dj_database_url
17
18 # Build paths inside the project like this: BASE_DIR / 'subdir'.
19 BASE_DIR = Path(__file__).resolve().parent.parent
20 # Load environment variables
21 env_file = BASE_DIR / 'env'
22 load_dotenv(dotenv_path=env_file)
23
24 # Quick-start development settings - unsuitable for production
25 # See https://docs.djangoproject.com/en/4.2/howto/deployment/checklist/
26
27 # SECURITY WARNING: keep the secret key used in production secret!
28 SECRET_KEY = 'django-insecure-alcftnjl1f9vuwkq5j1np43c8kxgl_u4k4hc+7g_l9d'
29 SECRET_KEY = os.environ.get("SECRET_KEY")
30
31 # SECURITY WARNING: don't run with debug turned on in production!
32 DEBUG = True
33 DEBUG = os.environ.get("DEBUG")
34
35 ALLOWED_HOSTS = ['*']
36
37 RPCAPIBASEURL = os.environ.get("RPCAPIBASEURL")
38
39 # Application definition
40
41 INSTALLED_APPS = [
42     'django.contrib.admin',
43     'django.contrib.auth',
44     'django.contrib.contenttypes',
45     'django.contrib.sessions',
46     'django.contrib.messages',
47     'django.contrib.staticfiles',
48     'visaAppRPCFrontend.apps.AppConfig',
49 ]
50
51 MIDDLEWARE = [
52     'django.middleware.security.SecurityMiddleware',
53     'django.contrib.sessions.middleware.SessionMiddleware',
54     'django.middleware.common.CommonMiddleware',
55     'django.middleware.csrf.CsrfViewMiddleware',
56     'django.contrib.auth.middleware.AuthenticationMiddleware',
57     'django.contrib.messages.middleware.MessageMiddleware',
58     'django.middleware.clickjacking.XFrameOptionsMiddleware',
59 ]
60
61 ROOT_URLCONF = 'visaSite.urls'
62
```

## Ejercicio número 6:

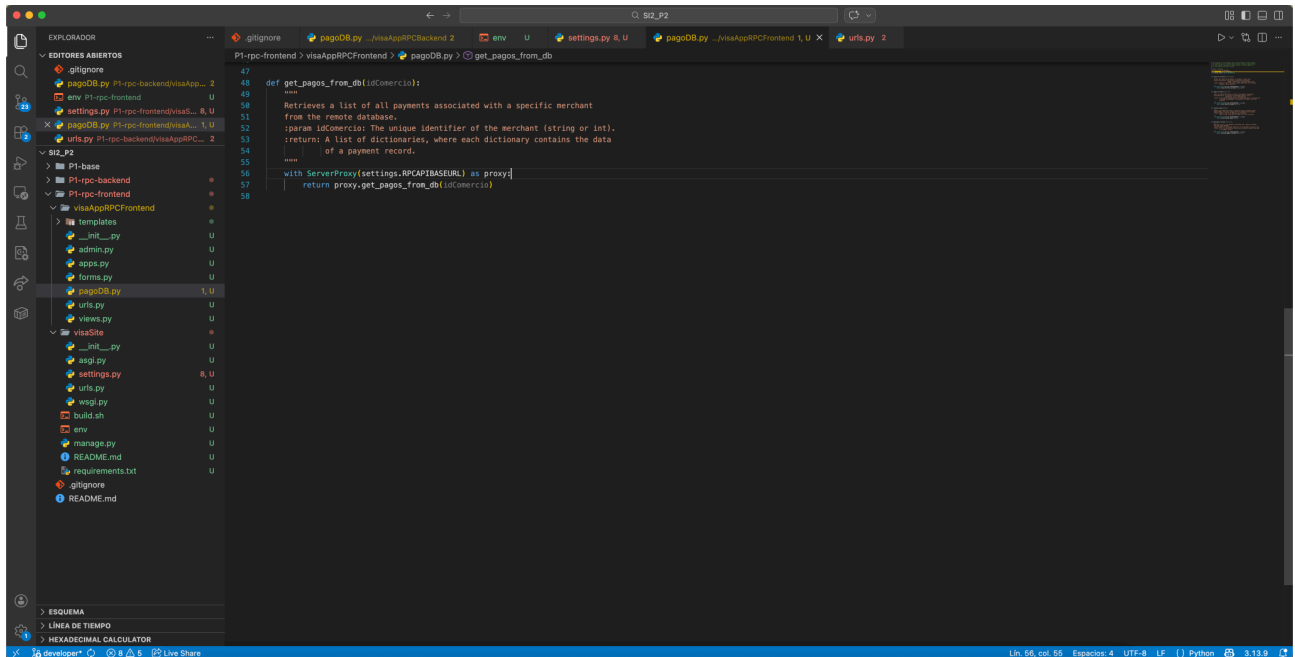
Ejecute los pasos descritos anteriormente, uno tras otro, para crear el nuevo fichero `pagoDB.py`, que invoque la funcionalidad de la aplicación como llamadas a procedimiento remoto. Comenta cada función describiendo tanto los argumentos de entrada como los valores devueltos. Incluya en la memoria evidencias (capturas de pantalla) de haber realizado estos pasos. Indica qué tipo de binding, de los tipos vistos en las transparencias de teoría sobre RPC (repasar dichas transparencias), realiza el frontend RPC codificado.

El frontend realiza un **binding estático**, ya que la dirección del endpoint RPC (`settings.RPCAPIBASEURL`) se resuelve en el momento de la configuración y no mediante un servicio de búsqueda de nombres o directorio en tiempo de ejecución.

Fichero `visaAppRPCFrontend/pagoDB.py`:



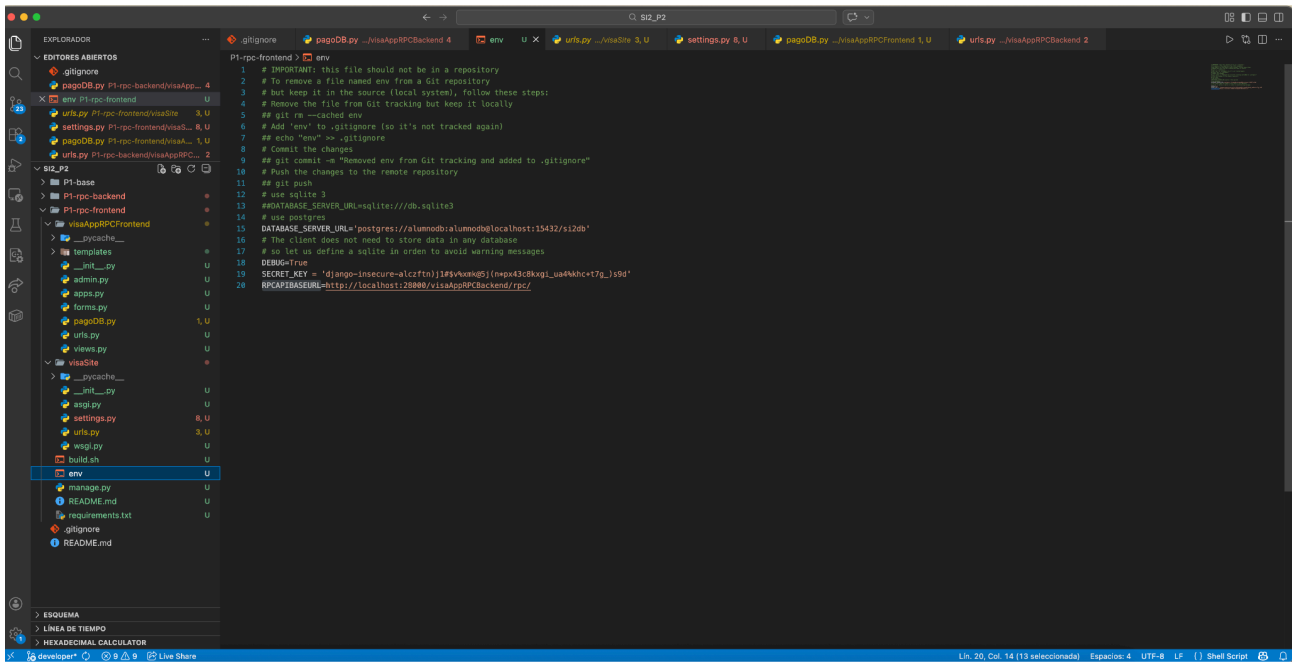
```
1 # This program is free software: you can redistribute it and/or modify
2 # it under the terms of the GNU General Public License as published by
3 # the Free Software Foundation, either version 3 of the License, or
4 # (at your option) any later version.
5
6 # author: rearabini
7 "Interface with the database"
8 from django.conf import settings
9 from xmlrpc.client import ServerProxy
10
11
12 def verificar_tarjeta(tarjeta_data):
13     """
14     Invokes the remote RPC procedure to validate a credit card.
15     :param tarjeta_data: A dictionary containing card details:
16         (number, holder_name, expiration_date, CVV).
17     :return: Returns True if the card is valid and exists in the remote
18             database, False otherwise.
19     """
20     with ServerProxy(settings.RPCAPIBASEURL) as proxy:
21         return proxy.verificar_tarjeta(tarjeta_data)
22
23
24 def registrar_pago(pago_dict):
25     """
26     Calls the remote RPC method to record a new payment transaction.
27     :param pago_dict: A dictionary with payment details such as:
28         (idComercio, price, idTarjeta_id).
29     :return: A dictionary representing the created Pago object (including the
30             timestamp 'marcaTiempo') if successful, or None if an error
31             occurred.
32     """
33     with ServerProxy(settings.RPCAPIBASEURL) as proxy:
34         return proxy.registrar_pago(pago_dict)
35
36
37 def eliminar_pago(idPago):
38     """
39     Requests the remote RPC server to delete a specific payment record.
40     :param idPago: The unique integer ID of the payment to be deleted.
41     :return: Returns True if the deletion was successful,
42             False if the record was not found or an error occurred.
43     """
44     with ServerProxy(settings.RPCAPIBASEURL) as proxy:
45         return proxy.eliminar_pago(idPago)
```



## Ejercicio número 7:

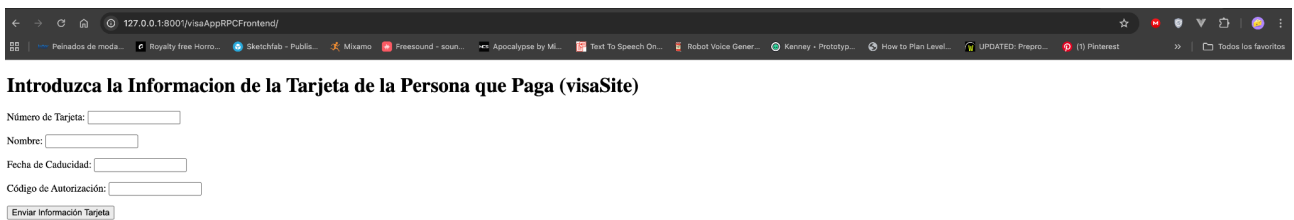
Ejecuta los pasos descritos más arriba e incluye evidencias en la memoria de haberlos llevado a cabo. Ejecuta los test proporcionados con el proyecto sobre las vistas y comprueba que no devuelven errores. Adjunta en la memoria una captura de pantalla en la que se muestre el resultado de ejecutar los test (python manage.py test).

## Fichero env del P1-rpc-frontend:



```
1 # WARNING: this file should not be in a repository
2 # To remove a file named env from a Git repository
3 # but keep it in the source (local system), follow these steps:
4 # Remove the file from Git tracking but keep it locally
5 ## git rm --cached env
6 # Add '.env' to .gitignore (so it's not tracked again)
7 ## echo ".env" >> .gitignore
8 # Commit the changes
9 ## git commit -m "Removed env from Git tracking and added to .gitignore"
10 # Push the changes to the remote repository
11 ## git push
12 # use sqlite 3
13 #DATABASE_SERVER_URL=sqlite:///db.sqlite3
14 # use postgres
15 DATABASE_SERVER_URL="postgres://alumno:alumno@localhost:5432/s12b"
16 # The client does not need to store data in any database
17 # so let us define a sqlite in order to avoid warning messages
18 DEBUG=True
19 SECRET_KEY = 'django-insecure-alcztft01f9vkvnm0j1nqx43c8xgl_uap4whcc+7g_js9'
20 RPCAPIBASEURL=http://localhost:28000/visaAppRPCBackend/rpc/
```

En el browser: <http://127.0.0.1:8001/visaAppRPCFrontend/>



**Introduzca la Informacion de la Tarjeta de la Persona que Paga (visaSite)**

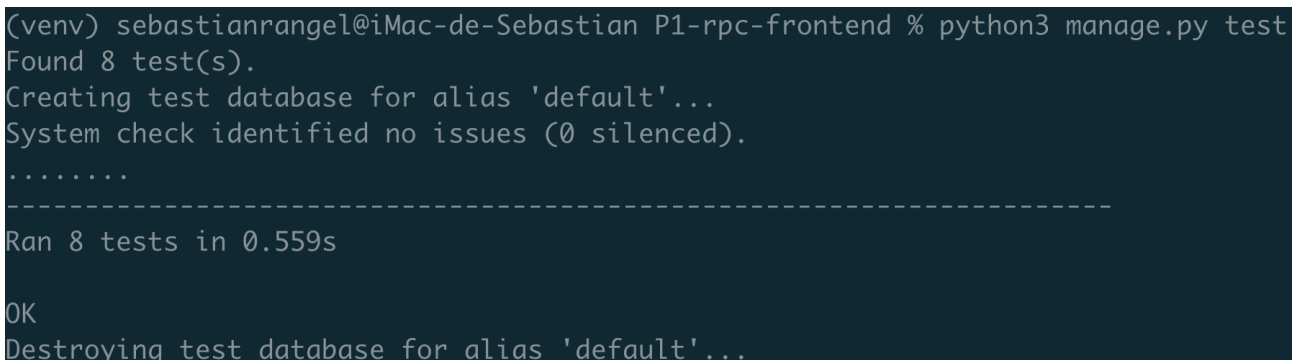
Número de Tarjeta:

Nombre:

Fecha de Caducidad:

Código de Autorización:

En el host:



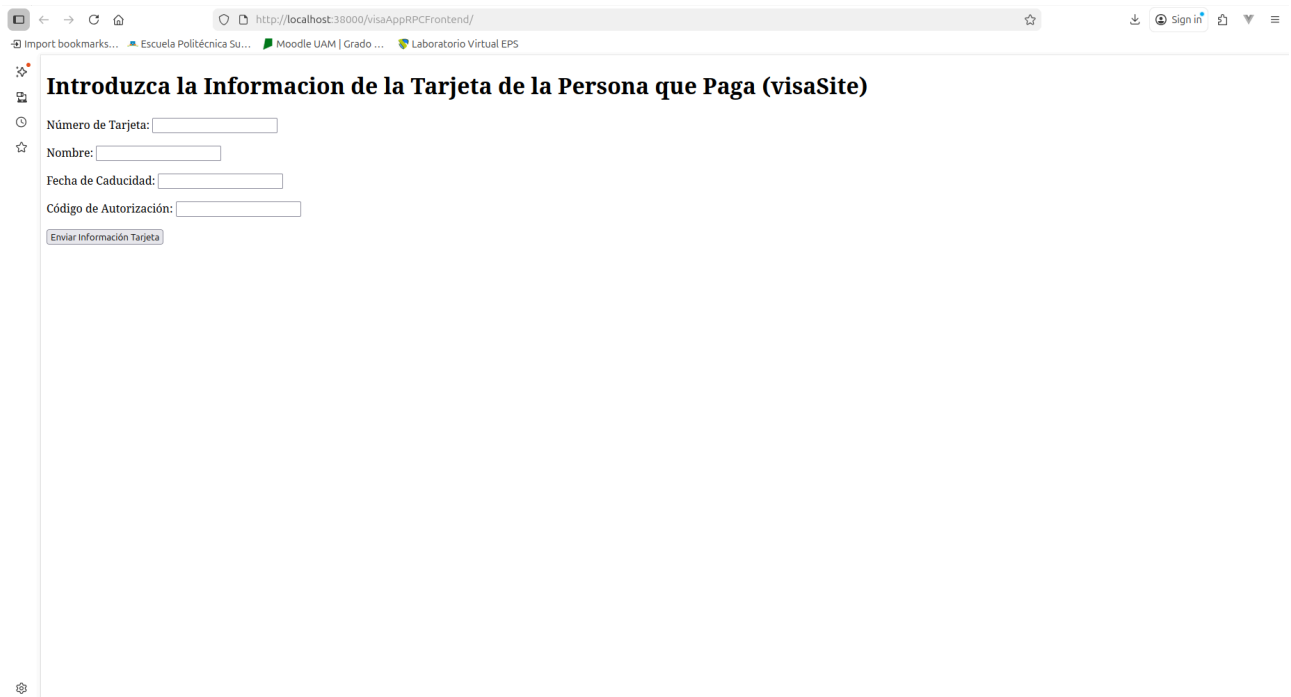
```
(venv) sebastianrangel@iMac-de-Sebastian P1-rpc-frontend % python3 manage.py test
Found 8 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 8 tests in 0.559s

OK
Destroying test database for alias 'default'...
```

## Ejercicio número 8:

Ejecuta los pasos descritos más arriba. Prueba a acceder desde el navegador del PC del laboratorio a la URL `localhost:38000/visaAppRPCFrontend/` para comprobar que el despliegue de la aplicación es correcto. Registra un pago, lístalo (tanto desde la aplicación usando testbd como desde un cliente SQL como DBeaver), y bórralo. Registra el pago tanto desde `localhost:38000/visaAppRPCFrontend/` como desde `localhost:38000/visaAppRPCFrontend/testbd`. Incluye evidencias en la memoria en forma de capturas de pantalla.

En el browser: <http://localhost:38000/visaAppRPCFrontend/>



Introduzca la Información de la Tarjeta de la Persona que Paga (visaSite)

Número de Tarjeta:

Nombre:

Fecha de Caducidad:

Código de Autorización:

## Despliegue del Frontend RPC en la VM3:

```
si2@si2-ubuntu-vm-3:~/repo/p1frontend.git$ sudo systemctl status gunicorn-frontend
● gunicorn-frontend.service - Gunicorn Frontend Service
   Loaded: loaded (/etc/systemd/system/gunicorn-frontend.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-03-02 12:40:46 CET; 37s ago
     Main PID: 2162 (gunicorn)
       Tasks: 2 (limit: 1670)
    Memory: 60.6M (peak: 60.9M)
       CPU: 507ms
    CGroup: /system.slice/gunicorn-frontend.service
            └─2162 /home/si2/venv/bin/python3 /home/si2/venv/bin/gunicorn --workers 1 --bind 0.0.0.0:8000 --pythonpath /home/si2/repo/p1fron>
              2173 /home/si2/venv/bin/python3 /home/si2/venv/bin/gunicorn --workers 1 --bind 0.0.0.0:8000 --pythonpath /home/si2/repo/p1fron>

mar 02 12:40:46 si2-ubuntu-vm-3 systemd[1]: Started gunicorn-frontend.service - Gunicorn Frontend Service.
mar 02 12:40:46 si2-ubuntu-vm-3 gunicorn[2162]: /home/si2/venv/lib/python3.12/site-packages/gunicorn/util.py:25: UserWarning: pkg_resources is
mar 02 12:40:46 si2-ubuntu-vm-3 gunicorn[2162]:   import pkg_resources
mar 02 12:40:46 si2-ubuntu-vm-3 gunicorn[2162]: [2026-03-02 12:40:46 +0100] [2162] [INFO] Starting gunicorn 20.1.0
mar 02 12:40:46 si2-ubuntu-vm-3 gunicorn[2162]: [2026-03-02 12:40:46 +0100] [2162] [INFO] Listening at: http://0.0.0.0:8000 (2162)
mar 02 12:40:46 si2-ubuntu-vm-3 gunicorn[2162]: [2026-03-02 12:40:46 +0100] [2162] [INFO] Using worker: sync
mar 02 12:40:46 si2-ubuntu-vm-3 gunicorn[2173]: [2026-03-02 12:40:46 +0100] [2173] [INFO] Booting worker with pid: 2173
lines 1-18/18 (END)
```

## Registro de pago desde <http://localhost:38000/visaAppRPCFrontend/>

←

→

🔍

localhost:38000/visaAppRPCFrontend/

☆

🔒

📄

🌐

⋮

📁

Peinados de moda...

Royalty free Horro...

Sketchfab - Públi...

Mixamo

Freesound - soun...

Apocalypse by Ml...

Text To Speech On...

Robot Voice Gener...

Kennay - Prototyp...

How to Plan Level...

UPDATED: Prepro...

(1) Pinterest

»

Todos los favoritos

### Introduzca la Informacion de la Tarjeta de la Persona que Paga (visaSite)

Número de Tarjeta:

Nombre:

Fecha de Caducidad:

Código de Autorización:

## Registro pago desde <http://localhost:38000/visaAppRPCFrontend/pago>

←

→

🔍

localhost:38000/visaAppRPCFrontend/pago/

☆

🔒

📄

🌐

⋮

📁

Peinados de moda...

Royalty free Horro...

Sketchfab - Públi...

Mixamo

Freesound - soun...

Apocalypse by Ml...

Text To Speech On...

Robot Voice Gener...

Kennay - Prototyp...

How to Plan Level...

UPDATED: Prepro...

(1) Pinterest

»

Todos los favoritos

### Registro de Pago (visaSite)

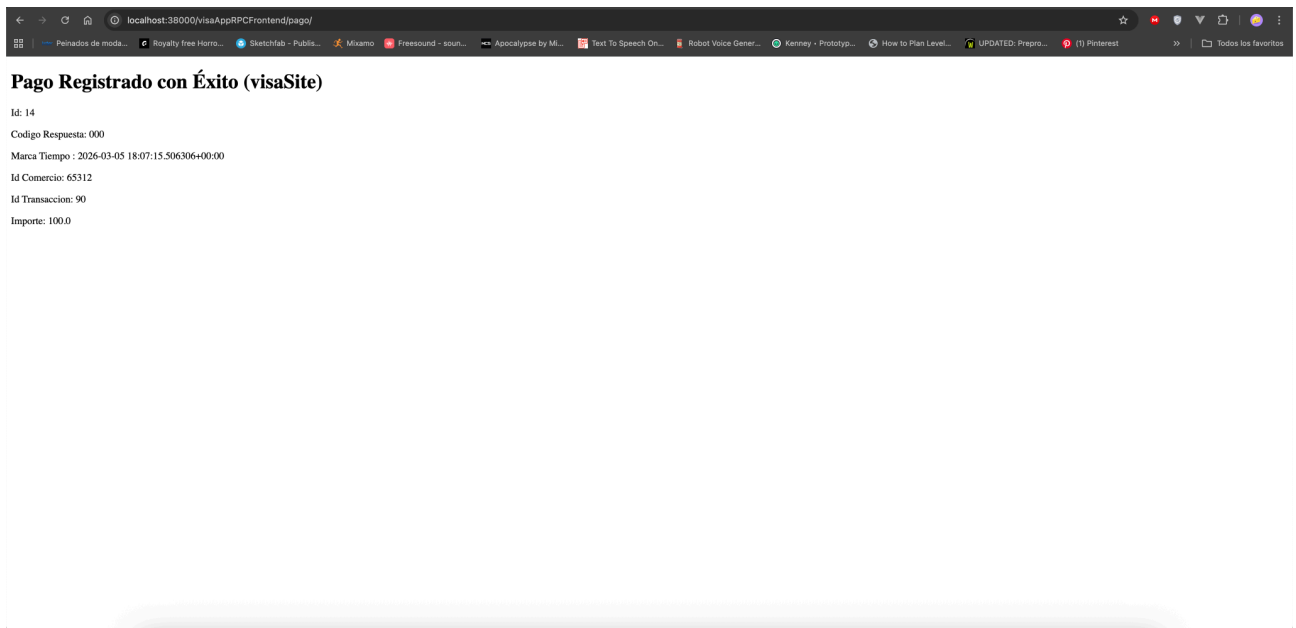
Introduzca los datos del nuevo pago a registrar:

ID Comercio:

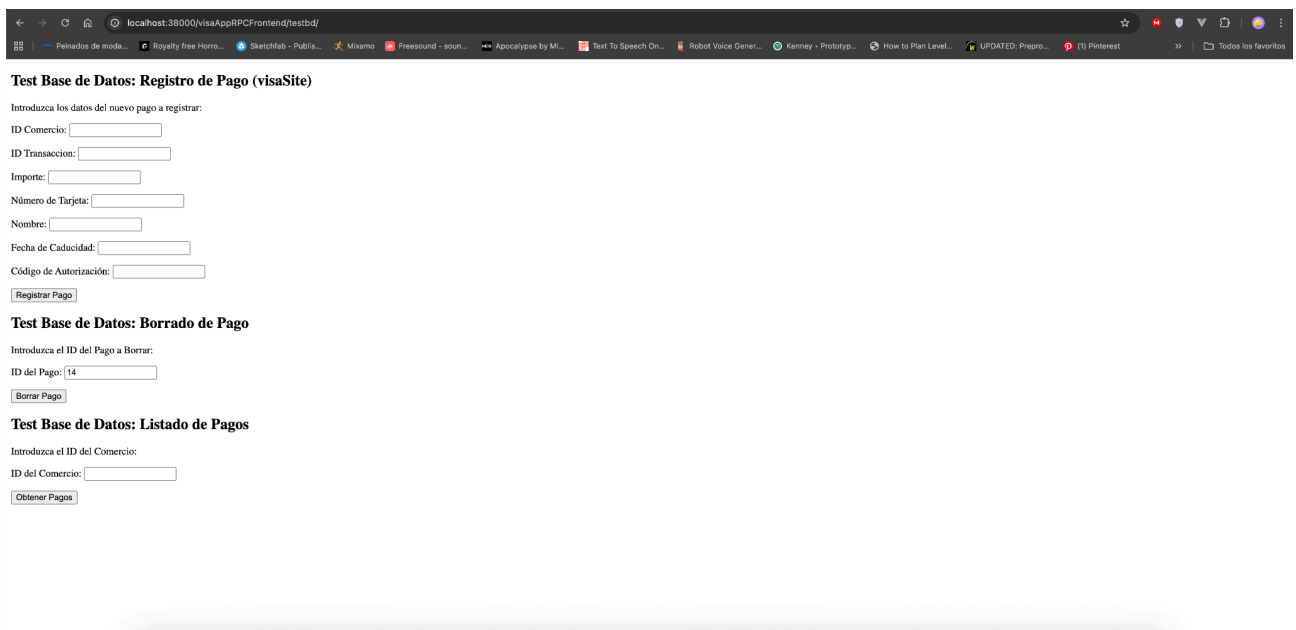
ID Transaccion:

Importe:

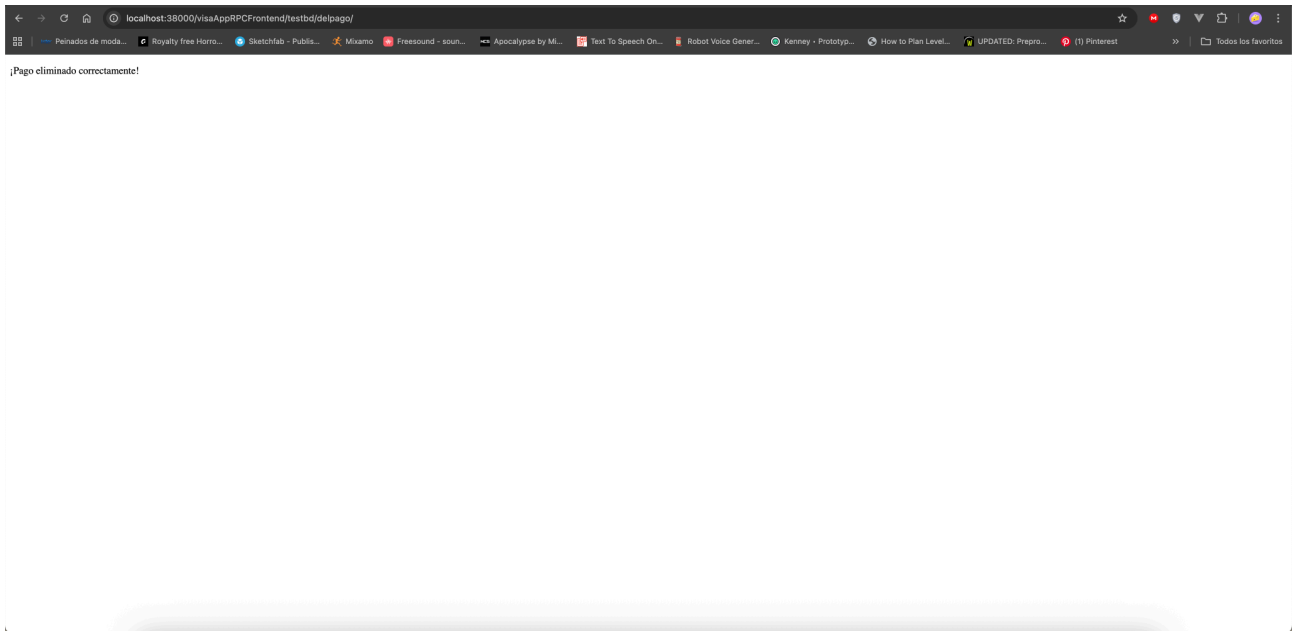
## Registro pago exitoso:



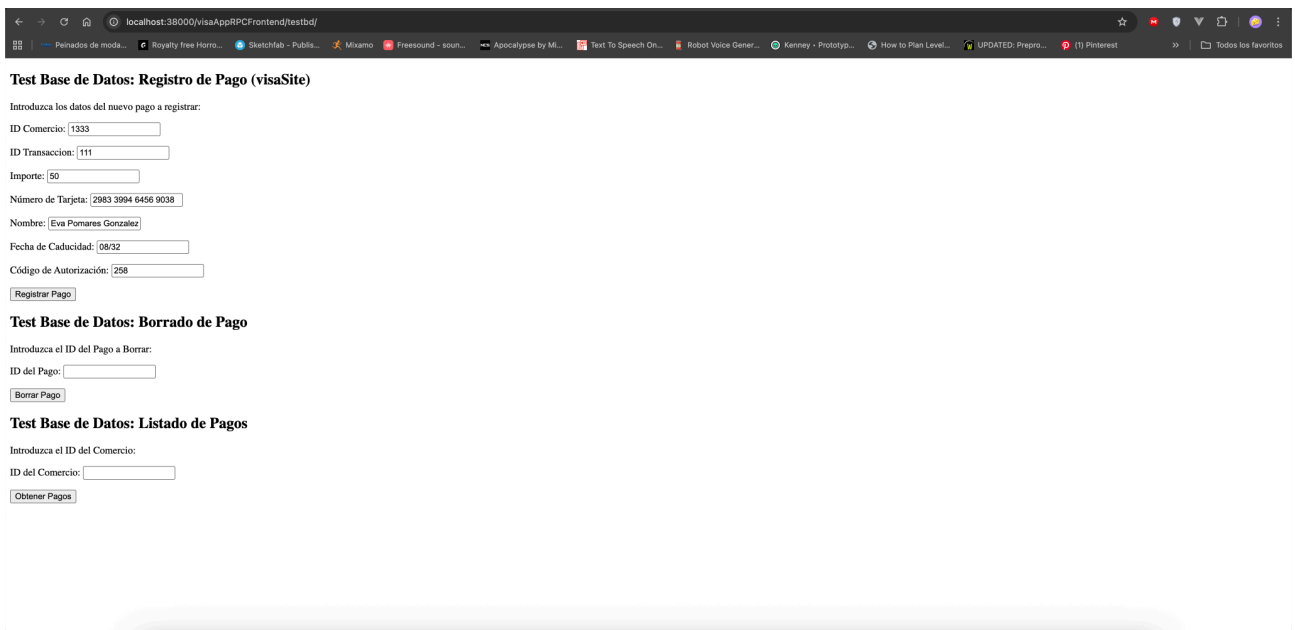
## Borrado pago id 14:



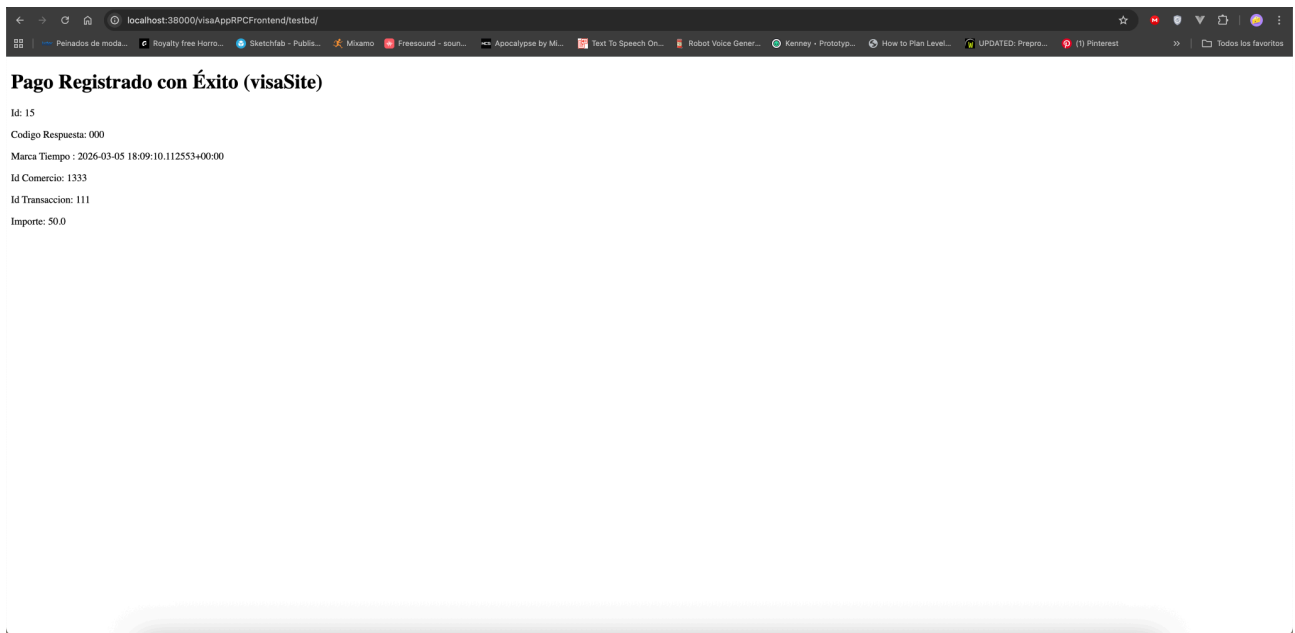
## Borrado pago (id 14) exitoso:



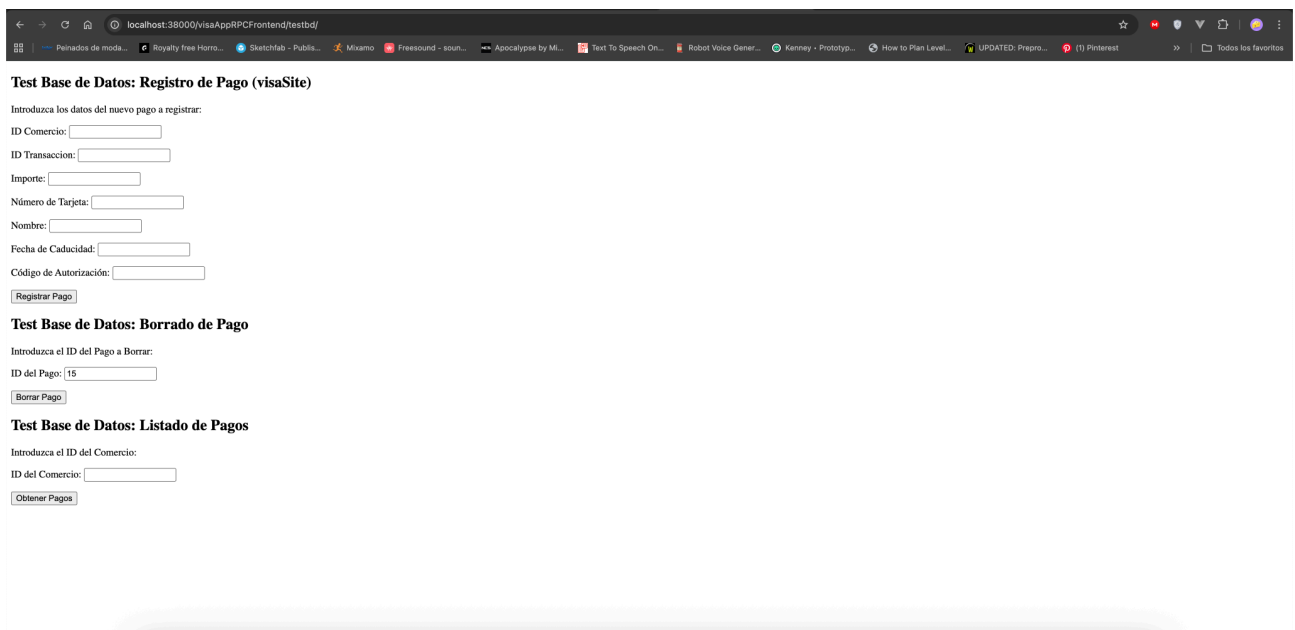
## Registro pago desde <http://localhost:38000/visaAppRPCFrontend/testbd>



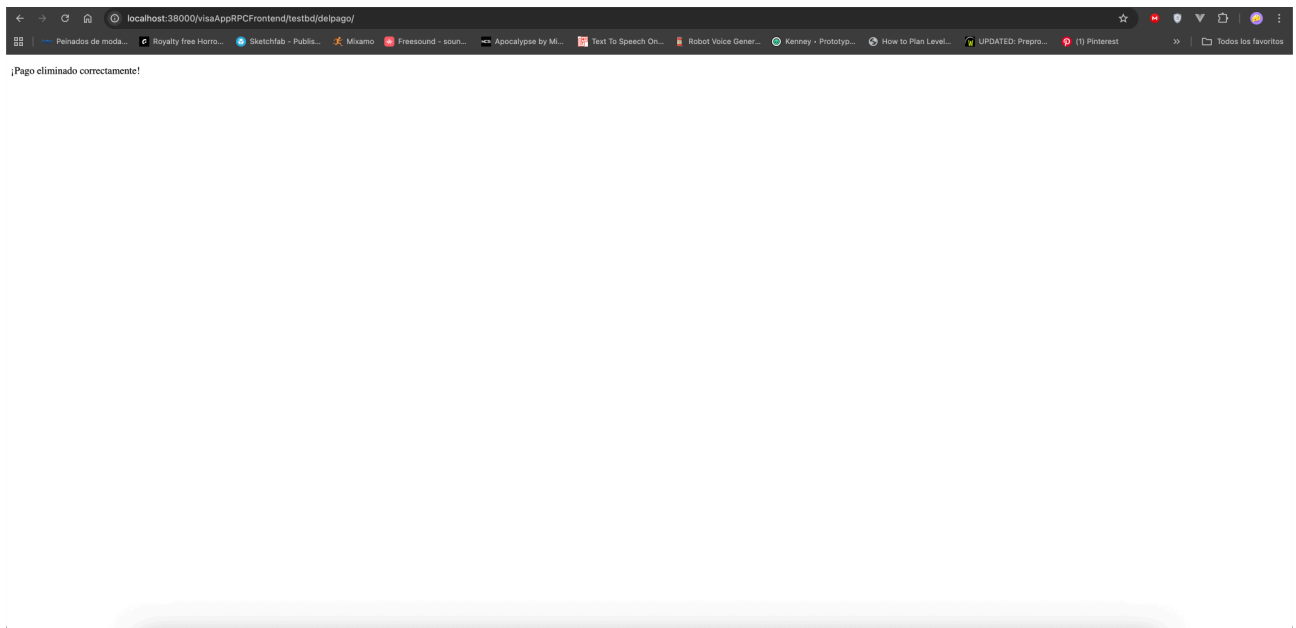
## Registro pago exitoso:



## Borrado pago id 15:

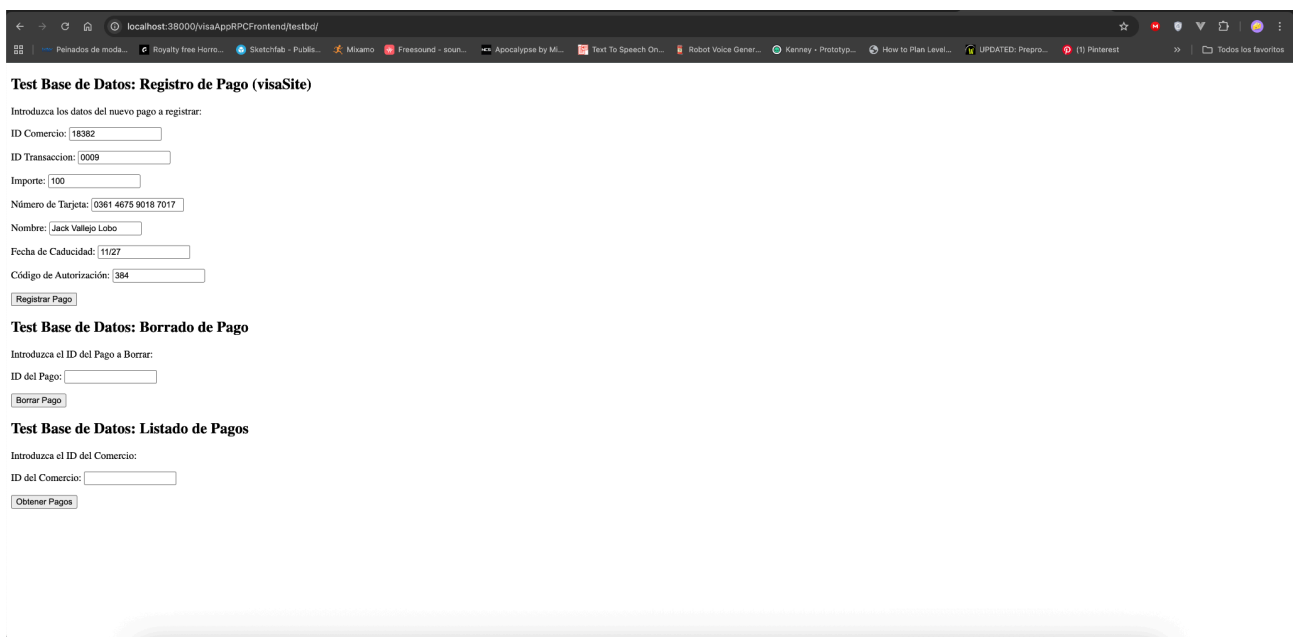


## Borrado pago (id 15) exitoso:

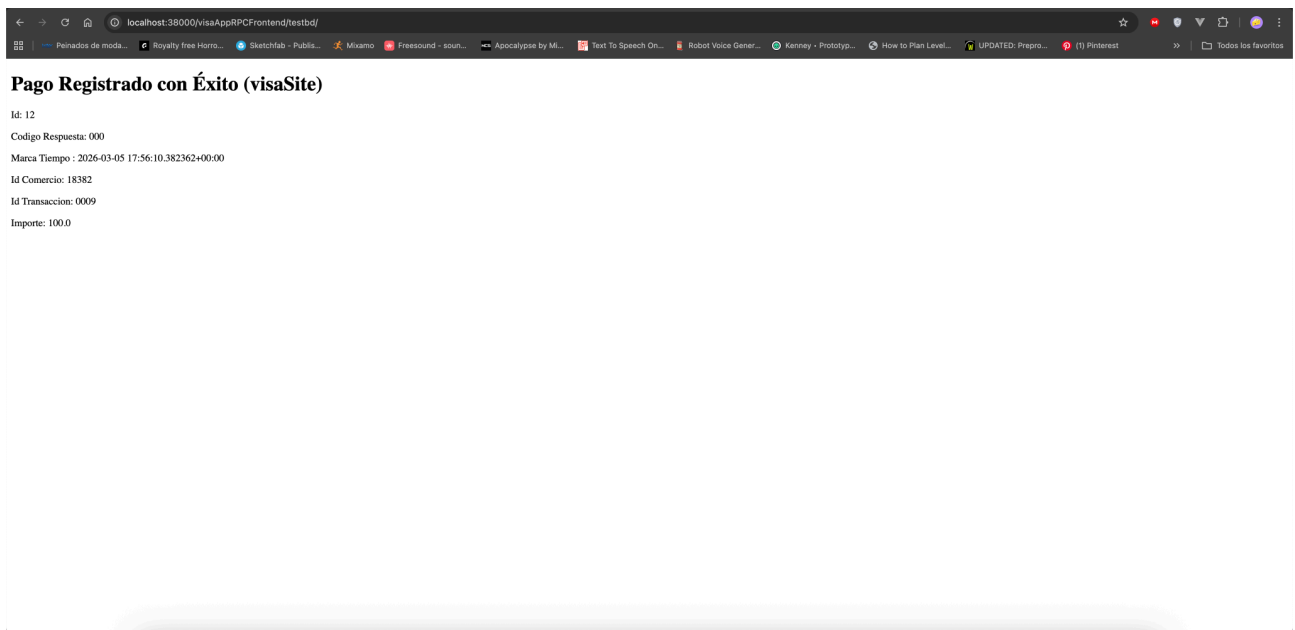


## Registro pago desde DBeaver

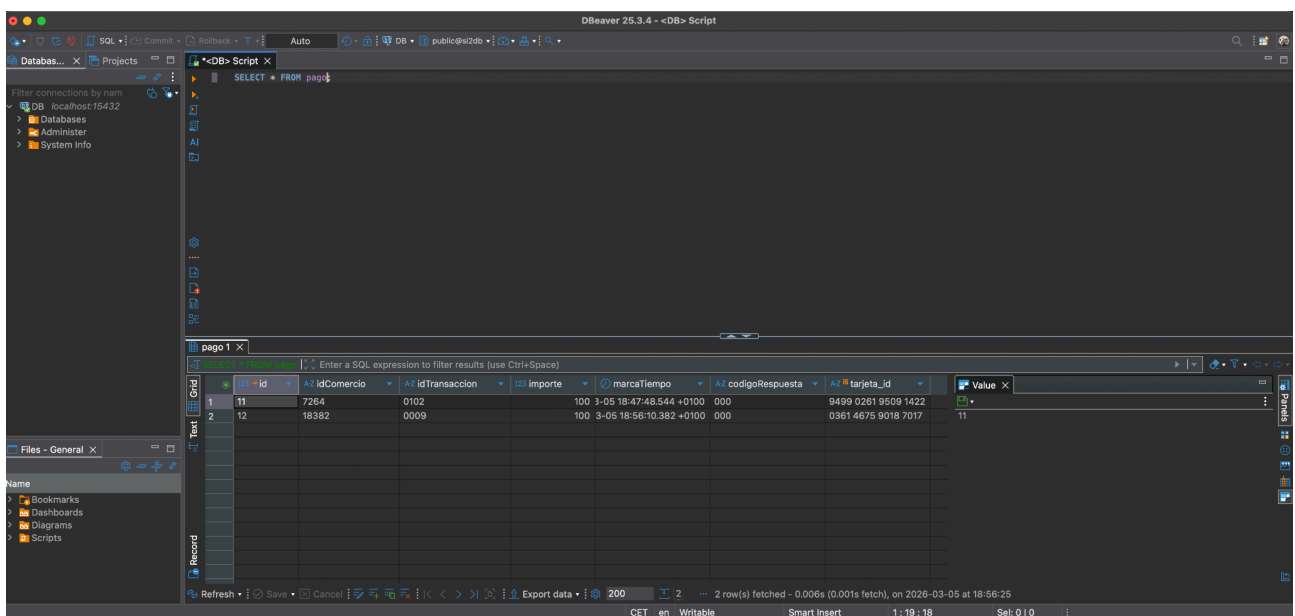
En el browser: <http://localhost:38000/visaAppRPCFrontend/testbd/>



## Registro pago exitoso:



## Comprobación registro en DBeaver:



*NOTA: Es el segundo pago, el primero lo había añadido con anterioridad por pruebas.*

## Borrado del pago:

localhost:38000/visaAppRPCFrontend/testbd/

Peinados de moda... Royalty free Horro... Sketchfab - Publi... Mixamo Freesound - soun... Apocalypse by M... Text To Speech On... Robot Voice Gener... Kenney - Prototyp... How to Plan Level... UPDATED: Prepro... (1) Pinterest Todos los favoritos

### Test Base de Datos: Registro de Pago (visaSite)

Introduzca los datos del nuevo pago a registrar:

ID Comercio:

ID Transaccion:

Importe:

Número de Tarjeta:

Nombre:

Fecha de Caducidad:

Código de Autorización:

### Test Base de Datos: Borrado de Pago

Introduzca el ID del Pago a Borrar:

ID del Pago:

### Test Base de Datos: Listado de Pagos

Introduzca el ID del Comercio:

ID del Comercio:

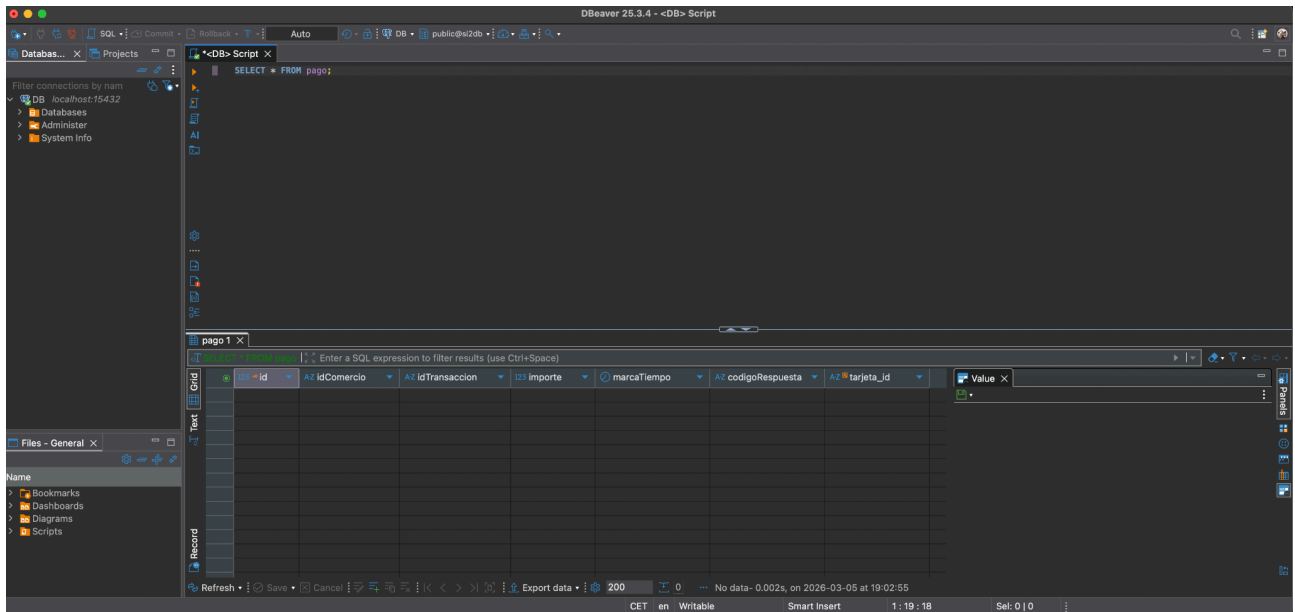
## Confirmación borrado:

localhost:38000/visaAppRPCFrontend/testbd/delpago/

Peinados de moda... Royalty free Horro... Sketchfab - Publi... Mixamo Freesound - soun... Apocalypse by M... Text To Speech On... Robot Voice Gener... Kenney - Prototyp... How to Plan Level... UPDATED: Prepro... (1) Pinterest Todos los favoritos

¡Pago eliminado correctamente!

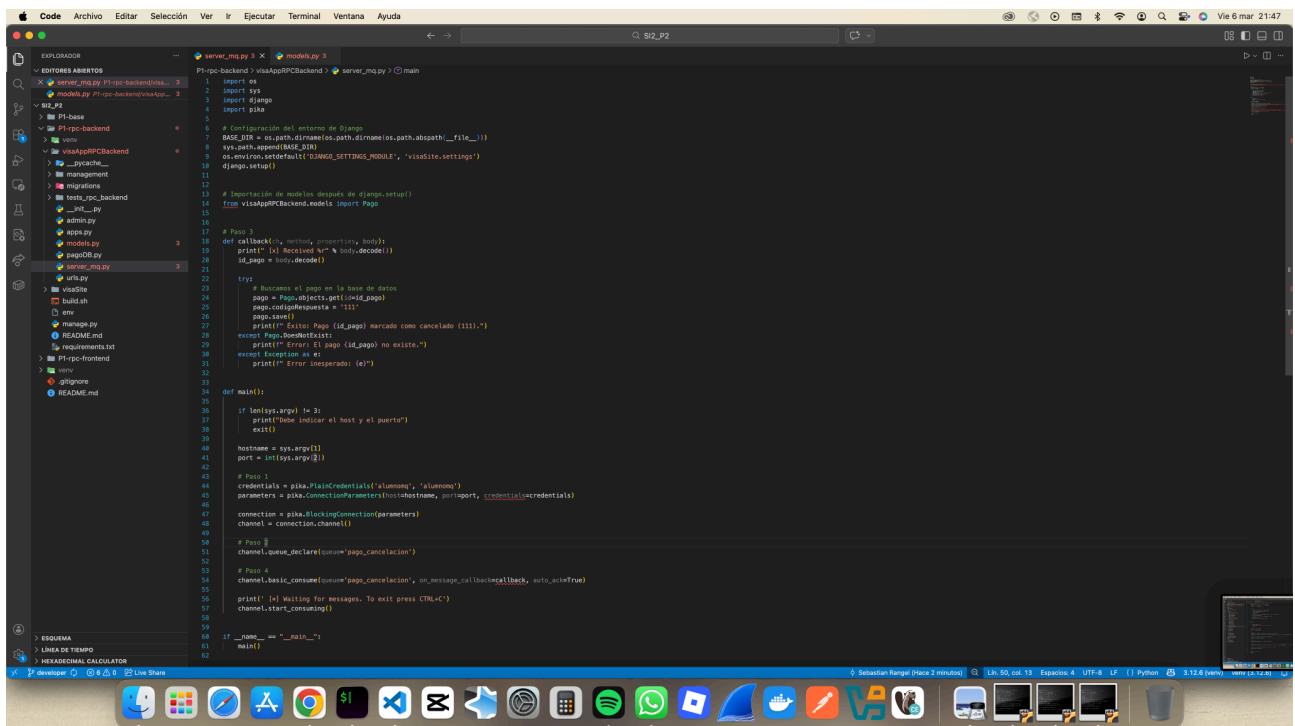
## Comprobación borrado en DBeaver:



## Ejercicio número 9:

Ejecuta los pasos descritos más arriba. Incluye evidencias en la memoria en forma de capturas de pantalla del código desarrollado.

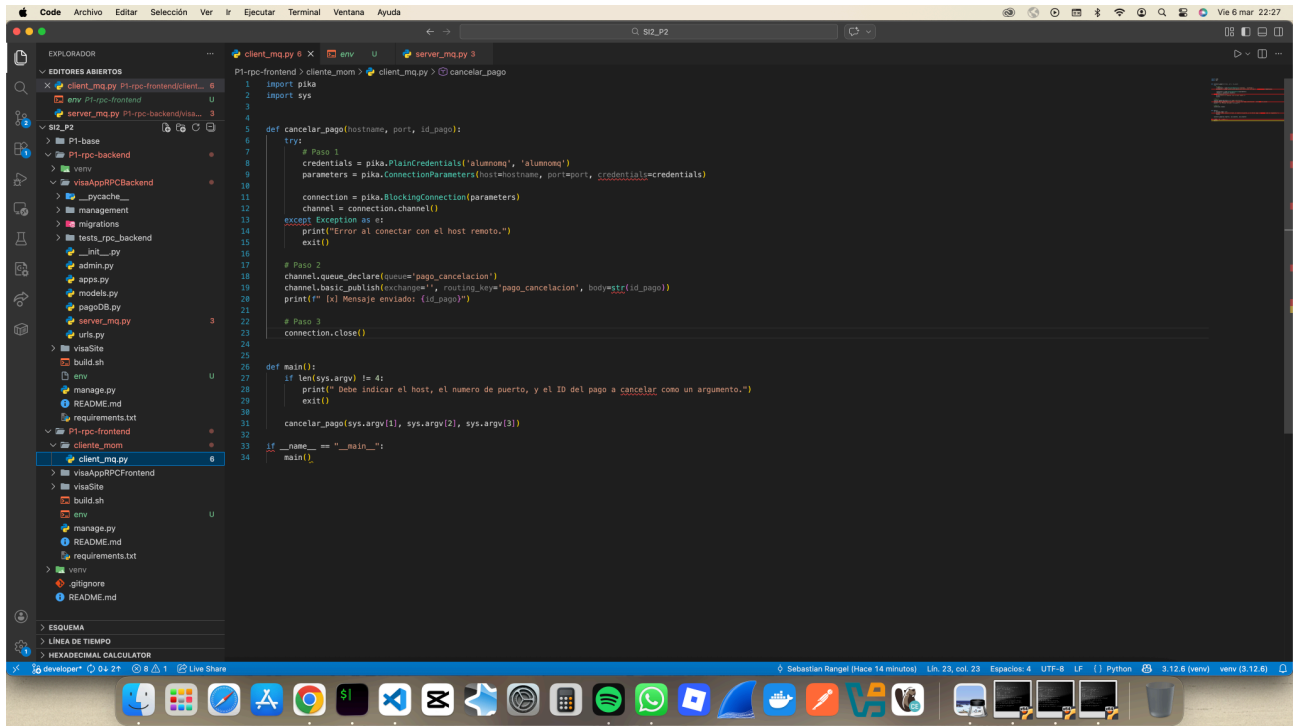
visaAppRPCBackend/server\_mq.py:



## Ejercicio número 10:

Ejecuta los pasos descritos más arriba. Incluye evidencias en la memoria en forma de capturas de pantalla del código desarrollado.

cliente\_mom/cliente\_mq.py:



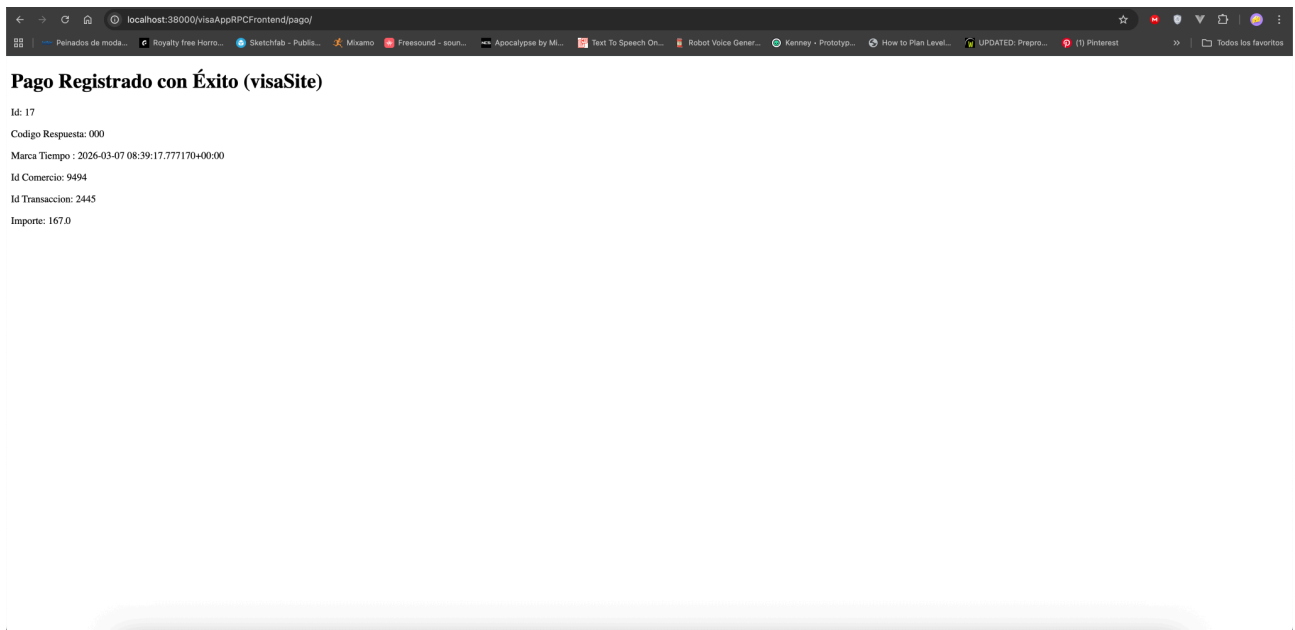
```
1 import pika
2 import sys
3
4
5 def cancelar_pago(hostname, port, id_pago):
6     try:
7         # Paso 1
8         credentials = pika.PlainCredentials('alumnoq', 'alumnoq')
9         parameters = pika.ConnectionParameters(hostname, port=port, credential=credentials)
10
11         connection = pika.BlockingConnection(parameters)
12         channel = connection.channel()
13     except Exception as e:
14         print("Error al conectar con el host remoto.")
15         exit()
16
17     # Paso 2
18     channel.queue_declare(queue='pago_cancelacion')
19     channel.basic_publish(exchange='', routing_key='pago_cancelacion', body=id_pago)
20     print(f"[x] Mensaje enviado: {id_pago}")
21
22     # Paso 3
23     connection.close()
24
25
26 def main():
27     if len(sys.argv) != 4:
28         print("Debe indicar el host, el numero de puerto, y el ID del pago a cancelar como un argumento.")
29         exit()
30
31     cancelar_pago(sys.argv[1], sys.argv[2], sys.argv[3])
32
33 if __name__ == "__main__":
34     main()
```

## Ejercicio número 11:

En este ejercicio probaremos a ejecutar el código desarrollado en los pasos anteriores.

**1. Ejecuta el cliente del servicio de cancelación de pagos en el PC del laboratorio. Para ello, escribir `python cliente_mq.py` seguido de la dirección IP, número de puerto de la VM1 para RabbitMQ, e identificador de pago a cancelar. De esta forma el servidor RabbitMQ que se usará será el de la VM1.**

Primero, registramos un pago en <http://localhost:38000/visaAppRPCFrontend>



Enviar el mensaje desde el host (en mi caso desde el iMac):

```
(venv) sebastianrangel@iMac-de-Sebastian cliente_mom % python3 client_mq.py localhost 5672 17
[x] Mensaje enviado: 17
(venv) sebastianrangel@iMac-de-Sebastian cliente_mom %
```

*Nota: Usamos localhost porque el túnel del iMac redirige ese 5672 a la VM1.*

**2. Como todavía no se ha arrancado el servidor de colas de mensajes, el mensaje enviado se almacenará en la cola. Probar a listar las colas y el número de mensajes en cada cola en la VM1 ejecutando el comando `sudo rabbitmqctl list_queues`. Comprobar que aparece la cola de mensajes asociada al servicio y que contiene un mensaje.**

Verificar la cola en la VM1:



```
si2@si2-ubuntu-vm-1:~$ sudo rabbitmqctl list_queues
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name      messages
pago_cancelacion    1
si2@si2-ubuntu-vm-1:~$
```

**3. Ejecuta el servidor del servicio de cancelación de pagos en la VM donde se esté ejecutando el servidor RPC. Debería ser la VM2. Para ello, escribir python servidor\_mq.py seguido de la dirección IP y número de puerto para RabbitMQ en la VM1. De esta forma usaremos el servidor RabbitMQ que se está ejecutando en la VM1.**

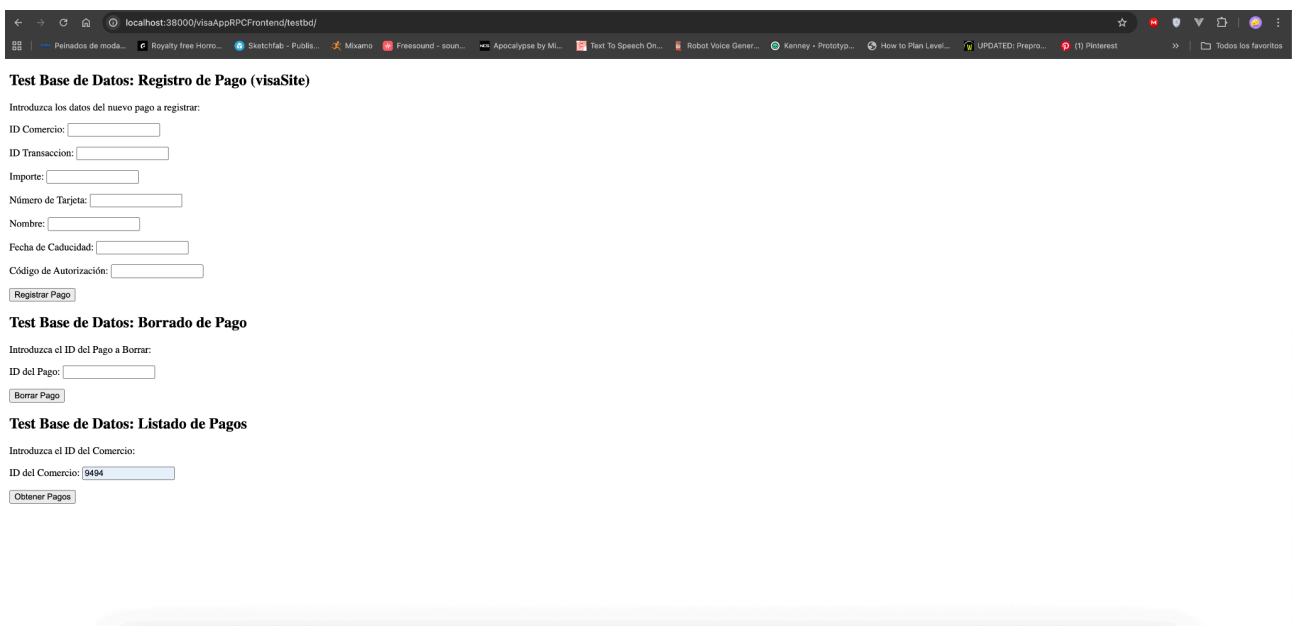
Ejecución del servidor en la VM2:

```
(venv) si2@si2-ubuntu-vm-2:~/repo/plbase/P1-rpc-backend/visaAppRPCBackend$ ls
admin.py  apps.py  __init__.py  management  migrations  models.py  pagoDB.py  __pycache__  server_mq.py  tests_rpc_backend  urls.py
(venv) si2@si2-ubuntu-vm-2:~/repo/plbase/P1-rpc-backend/visaAppRPCBackend$ python3 server_mq.py 192.168.1.135 5672
[*] Waiting for messages. To exit press CTRL+C
[x] Received '17'
éxito: Pago 17 marcado como cancelado (111).
```

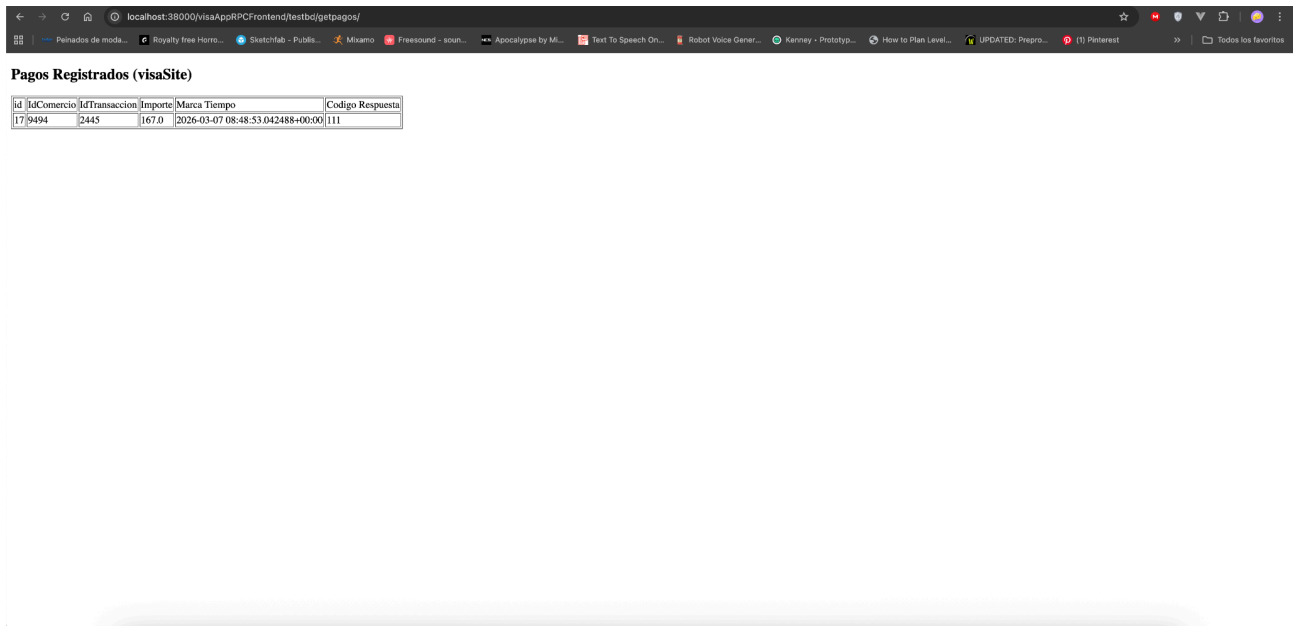
*Nota: Usamos la IP del iMac porque es nuestro puente hacia la VM1.*

**4. Comprobar que se ha cancelado el pago correspondiente en la base de datos. Para ello, probad a listar los pagos asociados a dicho comercio usando el frontend RPC ejecutándose en la VM3.**

Obtención del pago 17 en <http://localhost:38000/visaAppRPCFrontend/testbd>



Verificación del pago 17 y código de respuesta (111):



The screenshot shows a web browser window with the address bar displaying 'localhost:38000/visaAppRPCFrontend/testbo/getpagos/'. The page title is 'Pagos Registrados (visaSite)'. Below the title is a table with the following data:

| id | IdComercio | IdTransaccion | Importe | Marca Tiempo                     | Codigo Respuesta |
|----|------------|---------------|---------|----------------------------------|------------------|
| 17 | 9494       | 2445          | 167.0   | 2026-03-07 08:48:53.042488+00:00 | 111              |

5. Probar a listar las colas y el número de mensajes de cada cola, disponibles en la VM1, de nuevo. Comprobar que ya no aparecen mensajes almacenados en la cola.

Verificar limpieza de la cola en la VM1:

```
si2@si2-ubuntu-vm-1:~$ sudo rabbitmqctl list_queues
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages
pago_cancelacion    0
si2@si2-ubuntu-vm-1:~$
```

## Consideraciones finales.

Explicación del flujo de trabajo del compañero que trabaja con el sistema operativo macOS.

### 1. El iMac como "Hub" de Comunicaciones

Para permitir que estas máquinas se comuniquen entre sí y con el entorno de desarrollo local (iMac), se ha utilizado el **iMac como nodo central o "Bridge"** mediante el uso de túneles SSH con redirección de puertos y la opción de puerta de enlace (gateway).

### 2. Configuración del Entorno (.env)

Para garantizar la portabilidad y evitar errores, todos los archivos .env de las máquinas

(VM2, VM3) se han configurado para apuntar a la IP del iMac. Esto unifica el punto de acceso:

- DATABASE\_SERVER\_URL='postgres://alumnodb:alumnodb@192.168.1.135:15432/si2db'
- RPCAPIBASEURL=http://192.168.1.135:28000/visaAppRPCBackend/rpc/

### 3. Guía de comandos utilizados

#### 1. Túnel BBDD y MQ (hacia VM1):

```
ssh -g -L 15432:localhost:5432 -L 5672:localhost:5672 si2@localhost -p 12021
```

#### 2. Túnel Backend (hacia VM2):

```
ssh -g -L 28000:localhost:8000 si2@localhost -p 22022
```

- #### 3. Acceso Web Frontend (desde iMac):
- A través de <http://localhost:38000> (mapeado directamente al puerto 8000 de la VM3).